



DEBRE TABOR UNIVERSITY
GAFAT INSTITUTION OF TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE
Online House Rental Management System

In Partial Fulfillment of the Requirements for the Bachelor of Degree in Computer Science

Prepared by:

- | Name | Id.no |
|-----------------------------|---------|
| 1. Mahider Agmas----- | 0377/14 |
| 2. Mequannt Moges----- | 0385/14 |
| 3. Amahaeyesus Befirdu----- | 0351/14 |
| 4. Getaneh Kindu----- | 0364/14 |

Advisor Mr.G/rufael

Debre Tabor, Amhara, Ethiopia

JAN,2026

APPROVAL

The Project is our own and has not been presented for a degree in any other university with these functionalities and all the sources of material used for the project/thesis have been duly acknowledged.

Name	signature
1. Mahider Agmas	_____
2. Mequannt Moges	_____
3. Amahaeyesus Befirdu	_____
4. Getaneh Kindu	_____

This is to certify that I have read this project and that in my opinion it is fully adequate, in scope and quality, as a project for the degree of Bachelor of Science.

.....	
Name of Advisor	Signature

Examiner	Name	Signature
1. Examiner 1		
2. Examiner 2		
3. Examiner 3		
4. Examiner 4		

In-online House Rental Management System

Acknowledgment

First we would like to thank God for everything. And we would like to express our heartfelt thanks of gratitude to our advisor Mr. G/Rufael Wondie for guiding us in this project. We would also like to express our sincere gratitude to the entire department of Computer Science. Special thanks go to our friends who stood by us while working on this stuff.

Finally we express our deep sense of indebtedness to all those who have extended their support from time to time without whom the completion of this project was not possible.

Table of contents

APPROVAL	I
Acknowledgment	II
Table of contents	III
List of figures	VII
ABBREVIATION	IX
ABSTRACT	X
CHAPTER ONE	1
1. INTRODUCTION	1
1.1 Background	1
1.1.1 Vision	2
1.1.2 Mission	2
1.2 PROBLEM STATEMENT	2
1.3 Objective of the project	3
1.3.1 General Objective	3
1.3.2 Specific Objective	3
1.4 Feasibility Analysis	4
1.4.1 Operational Feasibility	4
1.4.2 Technical Feasibility	4
1.4.3 Economic Feasibility	4
1.4.4 Behavioral/Political feasibility	5
1.4.5 Schedule feasibility	5
1.5 Scope of the project	5
1.6 Limitation of the project	6
1.7 Significance of the project	6
1.8 Target beneficiaries of the Application	7
1.9 Methodology for the project	8
1.9.1 Fact Finding Techniques	9
1.10 Systems Analysis and Design approach	10
1.11 Development Tools	12
1.11.1 Tools and Techniques	12
1.11.2 Software Requirement	13
1.11.3 Hardware Requirement	14

1.12 Testing procedures	14
1.12.1 Unit Testing	14
1.12.2 Integration Testing	15
1.12.3 System Testing	15
1.13 Implementation	16
1.14 Risks & contingencies	16
1.14.1 Project Development problems	16
1.14.2 System problems on the USER	16
CHAPTER TWO	17
DESCRIPTION OF THE EXCITING SYSTEM	17
2. Introduction of Existing System	17
2.1 Introduction	17
2.2 existing system	17
2.3 organization structure	18
2.4 Services provided	19
2.5 Forms for the exciting system	21
2.6 Business rule identification	24
2.7 Online Payment System	30
Chapter Three	33
Software Requirement Specification	33
3. Introduction	33
3.1 General Constraints	33
3.2.1 External Interface Requirements	34
3.2.2 Use case Diagram	35
3.2.3 User Interfaces	36
3.2.4 Hardware Interfaces	39
3.2.4 Software Interfaces	39
3.2.5 Communications Interfaces	39
3.3 Functional Requirements	40
3.3.1 User Registration and Authentication	40
3.3.2 Listing and Management	40
3.3.3 Search and Viewing	40
3.3.4 Rental Request Management	40
3.3.5 Payment Management	40
3.3.6 Admin Management and Reporting	41
3.4. Use Case Design	41

3.4.1. Actor Identification	41
3.4.2. Use Case Identification	41
3.4.3. Use Case #1: Register Account	42
3.4.4. Use Case #2: List Property	42
3.4.5 Use Case Description	43
3.5 Sequence Diagram	44
3.6 Activity diagram	45
3.8 Class diagram	46
3.8.1 <class/object#1>	47
3.9 data structural model	49
3.9.1 Entity -relation ship diagram	50
3.10.Non-Functional Requirements	51
3.10.1. Performance	51
3.10.2 Reliability	51
3.10.3 Availability	52
3.10.4 Security	52
3.10.5 Maintainability	53
3.10.6 Portability	53
CHAPTER FOUR	55
SYSTEM DESIGN	55
4.1 Design Overview	55
4.2 System Architectural Design	56
4.2.1 Chosen System Architecture	56
4.2.2 System Interface Description	61
4.3 Detailed Description of Components	62
4.3.1 Authentication Component	62
4.3.2 Property Management Component	63
4.3.3 Rental Request Component	64
4.3.4 Payment Component	64
4.4 User Interface Design	65
4.4.1 Description of User Interface	65
4.4.2 Screenshot Images of the system	67
CHAPTER FIVE	75
SYSTEM TESTING	75
5.1 System Overview	75
5.2 Test Approach	75

Unit Testing	76
Integration Testing	76
System Testing	76
User Acceptance Testing (UAT)	76
Security Testing	76
Performance Testing	77
5.3 Test Plan	77
5.3.1 Features to be Tested	77
5.3.2 Features Not to be Tested	78
5.3.3 Testing Tools and Environment	78
5.4 Test Cases	79
5.5 Summary of Testing Results	80
CHAPTER SIX	81
CONCLUSION AND RECOMMENDATION	81
6.1 Conclusion	81
6.2 Recommendations	82
6.3 Future Work	83
References	85

List of figures

Figure 1 :Services provided diagram	20
Figure 2 :User Interaction Flow for Make Payment diagram	23
Figure 3 :Use Case diagram	36
Figure 4 :User Interface prototype diagram	38
Figure 5 :Sequence diagram	44
Figure 6 :Activity diagram	45
Figure 7 :Class diagram	46
Figure 8 : ER diagram	50

List of tables

Table 1 Table 1 Team Compositio	3
Table 2 Time schedule	5
Table 3 need for system upgrade	30
Table 4 proposed system solution	33
Table 5 User management	37
Table 6 property management	37
Table 7 search and interaction	37
Table 8 administration and payment	38
Error! Bookmark not defined. Table 9 actor identification	44
Table 10 Use case description	47

ABBREVIATION

Acronym	Meaning
OHRs	online house rental system
DB	user manage rule
UMR	Database
WBS	Work Breakdown Structure
SRS	Software Requirement Specification
FP	Functional Programming
UAT	User acceptance testing
UI	user interface
MVP	Minimum Viable Product
HTML	Hyper text markup language
Css	cascading style sheet

ABSTRACT

Finding a suitable rental house is often a time-consuming and inconvenient process, involving repeated visits, lack of proper information, and dependence on agents. The Online House Rental System is designed to simplify this process by providing a digital platform that connects house owners and tenants directly. This system allows property owners to list their rental houses with complete details such as location, rent, facilities, and availability. Tenants can search, view, and compare houses based on their preferences and contact owners without any middlemen. The system also includes user registration, secure login, and management features for administrators to monitor listings and users. By automating the rental process, the system reduces manual effort, saves time, and increases transparency. Overall, the Online House Rental System offers an efficient, user-friendly solution that makes house renting faster, easier, and more reliable for both owners and tenants..

CHAPTER ONE

1. INTRODUCTION

Finding a rental house is often a challenging and time-consuming task. Traditionally, people have to rely on visiting neighborhoods, asking friends or family, or searching through printed advertisements to find available houses. These methods are often inefficient, unreliable, and can be frustrating for both tenants and property owners.

An **Online House Rental System** provides a modern solution by creating a digital platform where property owners and tenants can connect easily. Property owners can list their houses online with important details such as rent, size, facilities, and location. Tenants can search, filter, and view these listings based on their specific needs, which makes finding a suitable house faster and more convenient.

This system not only saves time but also improves communication between landlords and tenants. It increases transparency by providing accurate and up-to-date information about available houses, helping tenants make informed decisions and assisting property owners in managing their listings efficiently. By using technology to simplify the house rental process, this system makes renting easier, more organized, and more accessible for everyone involved.

1.1 Background

Finding a house to rent has always been a difficult task for many people. In the traditional rental process, tenants often spend a lot of time visiting different areas, looking for available houses, or asking friends and family for recommendations. This process can be tiring, confusing, and sometimes leads to delays or poor decisions.

Similarly, property owners face challenges in finding reliable tenants for their houses. They often rely on advertisements, word of mouth, or physical notice boards to reach potential renters. This method is not only slow but also limits the reach to only a small number of people, which can leave houses vacant for a long time.

With the increasing use of technology, there is an opportunity to make the rental process easier and more efficient. An **Online House Rental System** provides a platform where property owners can list their houses online, and tenants can search for rental houses conveniently from their devices. This system helps to save time, reduce misunderstandings, and improve communication between landlords and tenants.

By introducing this system, the traditional challenges of house renting can be minimized, making the process more organized, transparent, and user-friendly for everyone involved.

1.1.1 Vision

To create a simple, reliable, and efficient platform that makes finding and renting houses easier for both tenants and property owners.

1.1.2 Mission

- To provide an online system where property owners can list rental houses with all necessary details.
- To help tenants search, compare, and select houses according to their needs quickly and conveniently.
- To improve communication between landlords and tenants and reduce misunderstandings in the rental process.
- To make the house rental process more organized, transparent, and user-friendly.

1.2 PROBLEM STATEMENT

The traditional house rental process has several challenges for both tenants and property owners:

1. Tenants spend a lot of time and effort visiting multiple locations to find a suitable house.
2. Information about available houses is often scattered and unreliable.
3. Tenants face difficulties in comparing houses and making informed decisions.

4. Landlords struggle to reach a wide number of potential tenants efficiently.
5. Houses may remain vacant for longer periods due to limited exposure.
6. Communication between tenants and landlords can be slow or lead to misunderstandings.
7. The overall rental process is time-consuming, inconvenient, and lacks transparency.

There is a need for an online system that allows landlords to list properties and tenants to search and select houses quickly, conveniently, and reliably.

TEAM COMPOSITION

Our group select democratic team composition to participate all members on all tasks of the project.

Table 1 Team Composition

no	Name	ID	Task
1	Mahider agmas	0377	Title selection, requirement analysis , gather information
2	Mequanint moges	0385	Title selection and gather information
3	Getaneh kindu	0364	Title selection and gather information
4	Amhaeyesus befirdu	0351	Title selection and gather information

1.3 Objective of the project

1.3.1 General Objective

To develop an online house rental system that simplifies the process of finding and renting houses for both tenants and property owners.

1.3.2 Specific Objective

- To provide a platform where property owners can list houses with details such as rent, size, and facilities.
- To enable tenants to search, filter, and view available houses based on their requirements.
- To improve communication between landlords and tenants and reduce misunderstandings.

- To save time and effort for both tenants and landlords in the house rental process.
- To make the rental process more organized, transparent, and user-friendly.

1.4 Feasibility Analysis

A feasibility study is conducted to determine whether the proposed online house rental system is practical, achievable, and beneficial. It examines different aspects including technical, economic, operational, and schedule feasibility.

1.4.1 Operational Feasibility

The system is designed to be user-friendly and easy to navigate. Landlords can upload property information quickly, and tenants can search, filter, and view listings easily. The system's interface and workflow are simple, ensuring that users with basic computer skills can operate it without difficulty.

1.4.2 Technical Feasibility

The system will be developed using commonly available technologies, including web development tools, database management systems, and programming languages. The hardware and software requirements are minimal and can be supported by standard computers and internet connections. Users only need basic devices such as computers, laptops, or smartphones with internet access, making the system technically feasible.

1.4.3 Economic Feasibility

Developing and implementing the system requires moderate cost for software development and hosting. However, the system will save time and reduce costs associated with traditional rental methods, such as traveling, advertising, or relying on intermediaries. In the long term, the system provides value by improving efficiency for both landlords and tenants that is seen instantly such as in purchasing products, paying employees etc.

1.4.4 Behavioral/Political feasibility

The system's success depends on users adapting to the online platform and following legal guidelines. Tenants and landlords benefit from faster, easier, and more organized transactions, while the system must remain user-friendly to encourage adoption. At the same time, it must comply with housing laws, rental regulations, and data privacy rules to ensure safe, reliable, and responsible use.

1.4.5 Schedule feasibility

This project is expected to be completed in two semesters of 2018 academic year. The plan of the project to be completed in this semester. But design, implementation, testing and maintenance would complete in the next semester including other minor activities. We will produce the project proposal and other activities of the project development based on the schedule shown below.

Table 2 Table for Time schedule

No	Task name	2018					
		Nov 15,2018	dec 30 ,2018	Jan 30,2018	March 15,2018	April 30,1018	may 20,2018
1	Title selection						
3	System Requirement specification						
4	System designing						
5	System implementing						
6	Operational testing						

1.5 Scope of the project

The scope of the Online House Rental System defines what the system will cover and the limits of its functionality.

- **Property Listings:** The system allows property owners to upload and manage information about rental houses, including details such as rent, size, facilities, and location.

- **Search and Filter:** Tenants can search for houses based on their preferences and filter results according to rent, size, or other features.
- **Communication:** The system facilitates easy communication between tenants and landlords through contact information provided in the listings.
- **User-Friendly Interface:** Both landlords and tenants can use the system easily, even with basic computer skills.
- **Data Management:** The system stores information about users and properties securely and allows updates whenever necessary.

1.6 Limitation of the project

- The system is designed for rental houses only and does not handle the sale of properties.
- It requires internet access and basic devices like computers or smartphones.
- Payment processing or legal agreement management is not included.

1.7 Significance of the project

The Online House Rental System provides several important benefits for both tenants and property owners:

- **Time and Effort Saving:** Tenants no longer need to visit multiple locations to find a house. They can search and compare options online, saving significant time and effort.
- **Improved Communication:** The system facilitates direct communication between tenants and landlords, reducing misunderstandings and delays.
- **Organized Information:** Property listings are centralized and easy to access, providing accurate and up-to-date information about available houses.
- **Convenience:** Users can access the system anytime and from anywhere with an internet connection, making the rental process flexible and efficient.
- **Transparency and Reliability:** The system reduces the chances of misinformation and helps tenants make informed decisions based on verified property details.

- **Support for Property Owners:** Landlords can reach a wider audience, manage their listings effectively, and reduce the time their houses remain vacant.
- ✓ generally, the project improves the house rental process by making it faster, easier, and more reliable, benefiting both tenants and property owners while encouraging the use of modern technology in everyday life.

1.8 Target beneficiaries of the Application

The Online House Rental System is designed to benefit multiple groups who are directly involved in the house rental process:

There are two main beneficiaries in our system. These are:

1. Tenants/Renters:

- Can search for rental houses easily and filter options according to their needs.
- Save time and effort by viewing details online instead of visiting multiple locations.
- Make informed decisions based on accurate and up-to-date property information.

2. Landlords/Property Owners:

- Can list houses online, reaching a larger audience than traditional methods allow.
- Manage and update property information conveniently.
- Reduce the time houses remain vacant by connecting quickly with potential tenants.
- The application primarily benefits tenants and landlords by making the rental process faster, simpler, and more reliable.

1.9 Methodology for the project

The methodology explains the approach and steps taken to design, develop, and implement the Online House Rental System.

1. Research and Requirement Gathering

- ❖ Conducted surveys and interviews with potential users (tenants and landlords) to understand their needs and challenges in the rental process.
- ❖ Studied existing rental systems and platforms to identify strengths, weaknesses, and features to include.
- ❖ Collected data on user expectations, preferred features, and common problems faced during renting.

2. System Design

- ❖ Developed system architecture to define how different components of the application interact, such as user interfaces, databases, and search functionalities.
- ❖ Created **data flow diagrams (DFD)** and **entity-relationship diagrams (ERD)** to model data storage and flow.
- ❖ Designed user-friendly interfaces for tenants and landlords to ensure easy navigation.

3. Development

- ❖ Used suitable programming languages, database management systems, and web technologies to build the system.
- ❖ Implemented core features such as property listing, search, filtering, and user registration.
- ❖ Focused on creating a secure, reliable, and responsive system.

4. Testing

- ❖ Performed **unit testing** to check individual functions of the system.
- ❖ Conducted **integration testing** to ensure all modules work together correctly.

- ❖ Gathered feedback from a small group of users to identify errors or areas for improvement.

5. Deployment and Implementation

- ❖ Deployed the system online so that tenants and landlords can access it easily.
- ❖ Provided guidance for users on how to register, list properties, and search for houses.

6. Maintenance and Updates

- ❖ Planned for regular updates to fix bugs, improve performance, and add new features based on user feedback.
- ✓ The project follows a systematic approach that starts from understanding user needs, designing a user-friendly system, developing and testing it, and finally deploying it for practical use. This methodology ensures that the system is efficient, reliable, and meets the objectives of simplifying the house rental process.

1.9.1 Fact Finding Techniques

Fact-finding techniques are methods used to collect information and understand the requirements for developing the Online House Rental System. Accurate and reliable information is essential to ensure the system meets the needs of tenants and landlords. The following techniques were used:

1.Interviews

- Conducted one-on-one discussions with tenants, landlords, and property managers.
- Gathered information about common problems, expectations, and desired features in a rental system.

2.Practical Observation

- Observed how tenants and landlords currently search for and manage rental properties.
- Noted challenges, delays, and inefficiencies in the traditional rental process.

1. Research

- Studied articles, reports, and case studies about online rental systems and housing management.
- Analyzed best practices and trends in digital property management.

By using these fact-finding techniques, comprehensive and accurate information was collected, which helped define the system requirements, design features, and functionalities. This ensures that the Online House Rental System effectively addresses the problems faced by tenants and landlords.

1.10 Systems Analysis and Design approach

The Systems Analysis and Design Approach outlines how the Online House Rental System is studied, planned, and developed to meet user needs efficiently. The approach ensures that the system is user-friendly, reliable, and capable of solving the problems faced in the traditional house rental process.

1. Systems Analysis

Systems analysis involves studying the current rental process to identify problems and define the requirements for the new system. The steps include:

- **Requirement Gathering:** Collecting information from tenants, landlords, and real estate agents using interviews, surveys, observation, and document review.
- **Problem Identification:** Recognizing inefficiencies such as scattered information, time-consuming property searches, and poor communication between tenants and landlords.
- **Requirement Specification:** Listing the functional and non-functional requirements of the system, such as property listing, search and filtering, communication features, and data security.

- **Feasibility Study:** Evaluating technical, economic, operational, and behavioral/political feasibility to ensure the project is practical and achievable.

2. Systems Design

Systems design focuses on planning the structure, interface, and functionality of the system based on the analysis. It includes:

- **Data Modeling:** Using Entity-Relationship Diagrams (ERD) to define the data structure for users, properties, and transactions.
- **Process Modeling:** Creating Data Flow Diagrams (DFD) to show how information flows between users, the database, and system modules.
- **Interface Design:** Developing user-friendly layouts for tenants and landlords to interact with the system easily.
- **Database Design:** Planning a secure and efficient database to store property details, user information, and communication logs.

3. Development Approach

The system follows a step-by-step development process to ensure reliability and quality:

- **Module-Based Development:** Building separate modules for property listing, search and filtering, user registration, and communication.
 - **Testing and Integration:** Performing unit testing on each module and integrating them to ensure smooth functionality.
 - **Deployment:** Making the system accessible online for real-world use.
 - **Maintenance and Updates:** Providing regular updates and improvements based on user feedback.
- ✓ By combining thorough systems analysis with careful design, the Online House Rental System is developed to be efficient, user-friendly, and capable of addressing the challenges of the traditional rental process. This structured approach ensures that both tenants and landlords benefit from a reliable and organized platform.

1.11 Development Tools

1.11.1 Tools and Techniques

1. Programming Languages

- **Node JS** – Which is JavaScript run time environment, Used to build the backend logic and implement system functions such as booking, user authentication, and payment processing.
- **HTML, CSS, JavaScript** – Used for the frontend interface.

2. Frameworks

- **React** – For building a responsive and user-friendly UI.

3. Database Tools

- **MongoDB** is a **NoSQL database** that stores data in a flexible, JSON-like format.

4. Communication & Collaboration Tools

- **Google Meet :**
- **Video calls**
- **Voice calls**
- **Screen sharing**
- **Chat messages**
- **Telegram**

❖ Techniques

1. Software Development Techniques

- **Functional Programming (FP)**
- **Structured Programming**

2. Requirement Gathering Techniques

- **Interviews**
- **Observation**
- **Document review**

- Surveys
- Focus group discussions

3. Testing Techniques

- Unit testing
- Integration testing
- System testing
- User acceptance testing (UAT)
- Regression testing

4. Project Execution Techniques

- **Agile Scrum** (sprints, daily stand-ups, sprint reviews, retrospectives)
- **Prototyping** – Early versions of the UI for feedback
- **Iterative development** – Continuous refinement of features

1.11.2 Software Requirement

- **Operating System:** Windows, Linux, or
- **Web Browser:** Google Chrome, Mozilla Firefox,
- **Web Server:** Node.js server for local development and testing
- **Database Management System (DBMS):** Node js.
- **Programming Languages:**
 - ✓ **Front-end:** HTML, CSS, JavaScript
 - ✓ **Back-end:** Node.js (depending on implementation)
- **Version Control:** Git and GitHub for source code management and collaboration
- **Database Management System (DBMS):** A DBMS is essential for storing and managing queue data, customer information, and system configurations. This include MySQL, PostgreSQL, MongoDB, or Web-based databases
- **Programming Languages:** HTML,CSS,java,java script
- **Operating System** Windows Server, Linux distributions (such as Ubuntu or Cent OS)

- **Browsers:** any browser
- **Web Server:** Apache, Nginx, or Microsoft Internet Information Services (IIS).
- **Microsoft office Word:** for documenting the corresponding deliverable associated with the project

1.11.3 Hardware Requirement

- **Computer:** -computer is a machine capable of doing many things. We use it to type on it and install all software and programming language.
- **Stationery:** Stationery like paper, pen and so on that were need. All these are necessary thing that we need to do and use.
- **Network cable:** since our system is web based, it is very necessary requirement. It is also help us to extract relevant information about our project from internet
- **Portable hard drive:** hard drives are small, lightweight, and easy to carry around.

1.12 Testing procedures

Testing is a critical step in the development of the Online House Rental System to ensure it works correctly, is reliable, and meets user requirements. The following testing procedures were applied:

1.12.1 Unit Testing

- ❖ Each module of the system, such as user registration, property listing, search and filtering, and communication, was tested individually.
- ❖ Ensured that each function works as expected and produces correct results.
- ❖ Example: Testing the property search function to make sure filters like rent, size, and facilities return accurate listings.

1.12.2 Integration Testing

- ❖ After unit testing, different modules were combined and tested to ensure they work together smoothly.
- ❖ Verified that data flows correctly between the user interface, database, and server.
- ❖ Example: Checking that a newly added property appears in the search results for tenants.

1.12.3 System Testing

- ❖ Tested the complete system as a whole to ensure it fulfills all functional and non-functional requirements.
- ❖ Focused on performance, usability, and security.
- ❖ Example: Ensuring that multiple users can use the system simultaneously without errors.

User Acceptance Testing (UAT)

- ❖ A small group of potential users (tenants and landlords) tested the system.
- ❖ Collected feedback on usability, interface design, and overall functionality.
- ❖ Example: Observing if tenants can easily search for houses and contact landlords without confusion.

5. Performance Testing

- ❖ Checked how the system performs under different conditions, such as multiple users accessing the system at the same time.
- ❖ Ensured that the system is responsive and does not crash under load.

6. Security Testing

- ❖ Verified that user data, including personal information and property details, is protected.
- ❖ Tested login authentication and restricted access to sensitive functions.

- ✓ By following these testing procedures, the Online House Rental System was verified to be reliable, user-friendly, secure, and efficient. Any bugs or errors identified during testing were corrected to ensure smooth operation before deployment.

1.13 Implementation

The proposed system will implement parallel with the existing system until the awareness of the workers towards the proposed system improved (parallel).

1.14 Risks & contingencies

1.14.1 Project Development problems

- **Unfortunate failure of system:** To handle this problem the teams have some method to resist not completely but partially by using back up mechanisms.
- **Power problem:** we tried to use laptops to cover the gap happened to our project during power failure.
- **Virus attack:** It is difficult to control data from virus but try to scan the data, installing and updating antivirus software and we use CDRW instead of flash.
- **Time management problem:** this problem by working cooperatively, divide our time by schedule for each phase of the project and we try to use the schedule effectively.

1.14.2 System problems on the USER

- **System failure:** To prevent system failure, utilize fault tolerance to handle failures gracefully, constantly monitor the system for signs of trouble, regularly test the system to identify and fix potential issues, and create a disaster recovery plan for how to recover the system in the event of a disaster.
- **Power failure:** Backup Power Generators: Installing backup power generators can provide an additional power source in the event of a power failure.

CHAPTER TWO

DESCRIPTION OF THE EXISTING SYSTEM

2. Introduction of Existing System

2.1 Introduction

In the existing house rental system, the process of renting is mostly handled manually or through informal platforms. Property owners usually advertise using newspapers, notice boards, word of mouth, or social media. Tenants must physically visit different locations to find available properties, collect information, and negotiate rental terms. This process is time-consuming, inefficient, and often unreliable.

Records related to tenants, rental agreements, and payments are commonly maintained on paper or basic spreadsheets, which increases the risk of data loss, duplication, and errors. Searching for suitable properties based on preferences such as location, rent amount, or type is difficult due to the lack of a centralized system. Communication between landlords and tenants is also unstructured and may lead to misunderstandings.

Additionally, the existing system lacks proper security, transparency, and tracking mechanisms. There is no easy way to verify property details, monitor rental history, or generate reports. Managing multiple properties and tenants becomes complex for landlords, while tenants face delays and limited options when searching for rentals. Due to these limitations, the existing system is inefficient and does not effectively meet the growing demand for a fast, organized, and reliable house rental management solution.

2.2 existing system

The existing house rental system is mostly manual and traditional. Property owners advertise rental properties through newspapers, notice boards, brokers, social media, or word

of mouth. Tenants search for s by visiting different locations or contacting agents, which requires a lot of time and effort.

All important information such as tenant details, house details, rental agreements, and payment records are maintained manually on paper or basic files. Because of this, data can be lost, duplicated, or incorrectly recorded. There is no centralized database to store or retrieve information easily.

Communication between landlords and tenants is unorganized, and rental requests are handled informally. There is no proper system to track availability, rental history, or payment status. Verifying property details and tenant information is also difficult.

2.3 organization structure

The organizational structure of a House Rental System defines the roles, responsibilities, and hierarchy involved in managing rental activities. The system is organized to ensure smooth coordination between administration, property owners, and tenants.

1. System Administrator

- ❖ Has the highest authority in the system
- ❖ Manages users (landlords and tenants)
- ❖ Verifies and approves property listings
- ❖ Maintains system security and data
- ❖ Generates reports and monitors system performance

2. Landlord / Property Owner

- ❖ Registers and logs into the system
- ❖ Adds, updates, and manages or property details
- ❖ Reviews rental requests from tenants
- ❖ Approves or rejects rental applications
- ❖ Manages rental agreements and availability status

3. Tenant / Renter

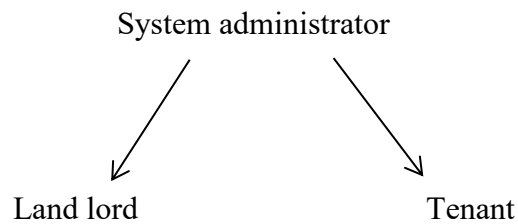
- ❖ Registers and logs into the system

- ❖ Searches for available rental s
- ❖ Views details and rental terms
- ❖ Sends rental requests to landlords
- ❖ Manages personal profile and rental history

4. Support / Maintenance (Optional)

- ❖ Provides technical support
- ❖ Handles system maintenance and updates
- ❖ Resolves user issues and system errors

Organizational Flow (Text Representation)



The organizational structure ensures proper control, accountability, and efficient management of the rental process. Each role has defined responsibilities, allowing the system to function smoothly and transparently.

2.4 Services provided

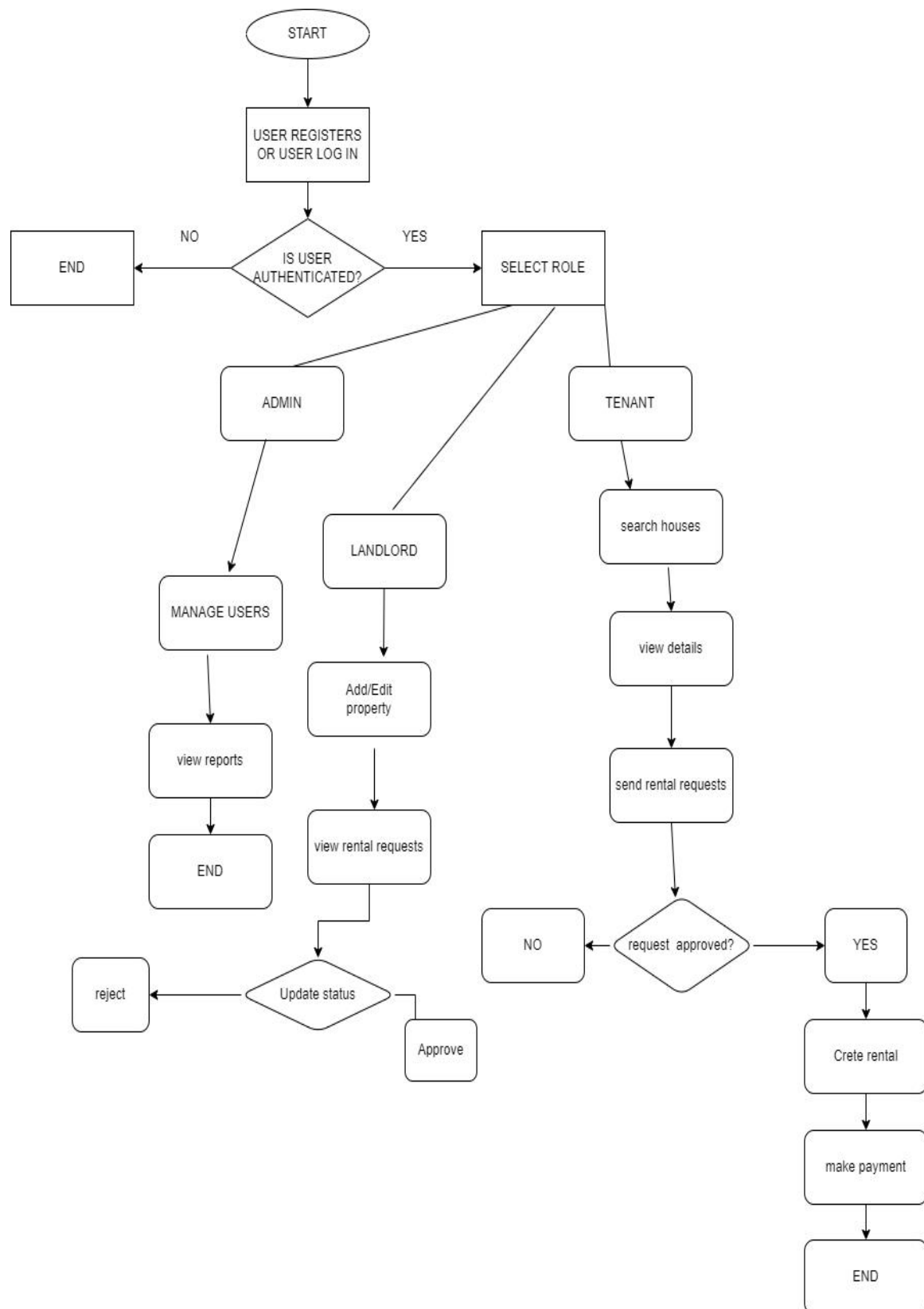


Figure 1:Services provided diagram

Simple Explanation of Flow

- 1) User logs into the system
- 2) System verifies user credentials
- 3) User selects role-based activities
- 4) Tenant searches and requests a
- 5) Landlord approves or rejects the request
- 6) Rental is confirmed
- 7) Payment is made (if applicable)
- 8) Process ends

2.5 Forms for the exciting system

These forms are designed to collect specific information from users, facilitate their interactions with the system, and streamline the queue management process. Here are some common forms that may be included in a web-based queue management system.

- **Registration Form:** Describe the registration form that users will need to fill out to create an account in the system. Include details about the required fields, validation rules, and any additional information that users need to provide.
- **Queue Number:** Patients are required to wait for extended periods while holding their assigned queue number upon arrival.
- **The Carding System:** The carding system poses a challenge as individuals may misplace their card due to forgetfulness or accidental circumstances, resulting in the inability to receive treatment.

2.5.1 Users

In a Rental System, different users interact with the system, but for the “Make Payment” service, the main users are the following:

1. Tenant (Primary User)

- ❖ The **tenant** is the main user of the *Make Payment* feature
- ❖ Pays monthly rent or advance amount
- ❖ Selects payment method (online/offline, if available)
- ❖ Views payment history and receipts
- ❖ Ensures rent is paid on time

2. Landlord / Property Owner (Secondary User)

- ❖ Receives rent payments from tenants
- ❖ Views payment status (paid / pending)
- ❖ Tracks payment history for each property
- ❖ Confirms or verifies payments (if manual confirmation is required)

3. System Administrator (Monitoring User)

- ❖ Monitors all payment transactions
- ❖ Manages payment-related issues or disputes
- ❖ Generates payment reports
- ❖ Ensures system security and data accuracy

User Interaction Flow for Make Payment

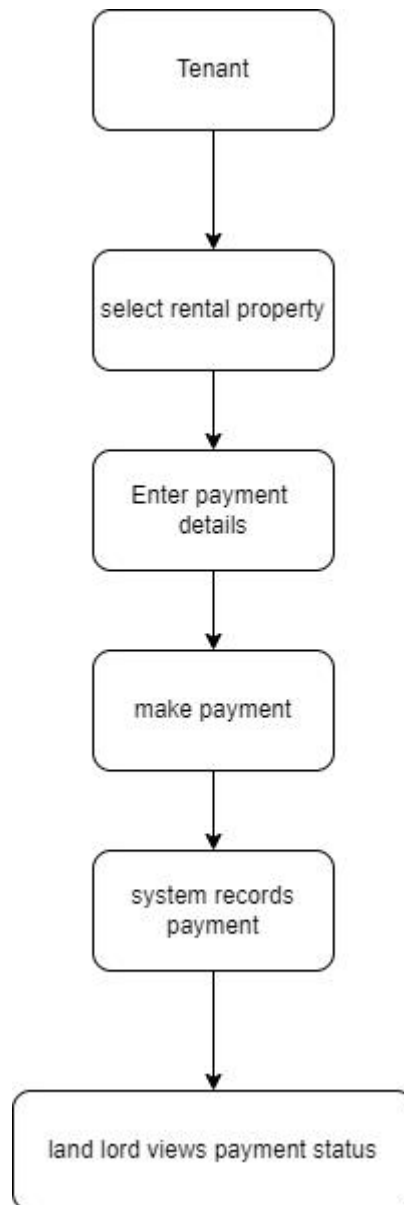


Figure 2:User Interaction Flow for Make Payment diagram

- ✓ The **Tenant** is the main user who performs the *Make Payment* action, while the **Landlord** receives and verifies the payment, and the **Admin** supervises and manages payment records.

2.6 Business rule identification

- Business rules define the **policies, constraints, and conditions** under which the Rental System operates. These rules ensure the system functions correctly, securely, and consistently for all users.

User Management Rules(UMR)

- ❖ Only registered users can access system features.
- ❖ A user must have one role only: Admin, Landlord, or Tenant.
- ❖ Email addresses must be unique for each user.
- ❖ Users must log in using valid credentials.
- ❖ Passwords must meet minimum security requirements.
- ❖ Admin accounts can only be created by existing admins.
- ❖ Users can update their profile information but not their role.

Property Management Rules

- ❖ Only landlords can create property listings.
- ❖ Each property must be associated with exactly one landlord.
- ❖ A property must include mandatory details:
 - Location
 - Rental price
 - Property type
 - Description
 - At least one image
- ❖ A property can only be listed as Available, Booked, or Unavailable
- ❖ A landlord can edit or remove only their own property listings.
- ❖ Properties marked as *Booked* cannot be rented by another tenant.

- ❖ Admins can remove any property that violates system policies.

Rental & Booking Rules

- ❖ Only tenants can request to rent a property.
- ❖ A tenant can request only one booking per property.
- ❖ A tenant cannot book multiple properties at the same time.
- ❖ Rental requests must be approved or rejected by the landlord.
- ❖ A rental request expires if not acted upon within a defined time period.
- ❖ Once approved, the property status changes to Booked.
- ❖ Rejected or expired requests cannot be reactivated.
- ❖ Tenants can cancel a pending booking request.
- ❖ Landlords cannot approve their own booking requests.

Payment Rules

- ❖ Rent payment must be completed before move-in.
- ❖ Payment confirmation is required to finalize booking.
- ❖ Payment records cannot be deleted, only archived.
- ❖ Refunds can only be initiated by an admin.
- ❖ Late payments may result in automatic booking cancellation.

Search & Access Rules

- ❖ Tenants can search properties by:
 - Location
 - Price range
 - Property type
- ❖ Only available properties appear in search results.
- ❖ Contact details of landlords are visible only to logged-in tenants.

Existing infrastructure

The existing infrastructure refers to how rentals are traditionally managed before

Introducing a computerized or online system.

1. Manual Infrastructure (Traditional System)

1.1 Physical Components

- ❖ Paper-based records (agreements, tenant details)
- ❖ Files and registers
- ❖ Notice boards and posters
- ❖ Physical office or rental agency

1.2 Human Resources

- ❖ Property owners (landlords)
- ❖ Tenants
- ❖ Brokers/agents
- ❖ Office staff

1.3 Communication Methods

- ❖ Phone calls
- ❖ Face-to-face meetings
- ❖ Printed advertisements
- ❖ Word-of-mouth

2. Existing Process Flow

- ❖ Landlord advertises property manually
- ❖ Tenant searches by visiting locations or agents
- ❖ Tenant contacts landlord or broker
- ❖ inspection in person
- ❖ Paper rental agreement signing
- ❖ Rent paid in cash or bank deposit
- ❖ Records maintained manually

3. Existing Technical Infrastructure (Semi-Digital)

Some systems use basic technology:

- ❖ Desktop computers
- ❖ Spreadsheet software (Excel)
- ❖ Basic databases
- ❖ Email communication
- ❖ Manual payment tracking

4. Limitations of Existing Infrastructure

- ❖ Time-consuming
- ❖ Error-prone record keeping
- ❖ No centralized database
- ❖ Difficult to search s
- ❖ Limited security
- ❖ No real-time availability updates
- ❖ High dependency on agents

5. Challenges in the Existing System

- ❖ Loss of records
- ❖ Fraud risk
- ❖ No transparency
- ❖ Poor communication
- ❖ Difficult report generation
- ❖ Limited accessibility

6. Need for System Upgrade

Due to these issues, an automated Rental System is needed to:

- ❖ Digitize records
- ❖ Provide online access
- ❖ Improve security
- ❖ Speed up searching
- ❖ Enable online payments
- ❖ Reduce manual workload

Table 3 need for system upgrade

Aspect	Existing infrastructure
Data storage	Paper files/excel
Search method	Manual visits
payments	Cash/bank deposit
Security	low
scalability	poor

Hardware Infrastructure

- ❖ Central server (local or cloud-based)
- ❖ Client devices (computers, tablets, Smartphone's)
- ❖ Network infrastructure (LAN / Internet)

Purpose:

Allows users (tenants, landlords, admins) to access the system through a web browser.

Software Infrastructure

1. Server Software

- ❖ Web server (Apache / Nginx)
- ❖ Operating System (Linux / Windows Server)

2 . Application Software

- ❖ Back end application (Node.js)
- ❖ Frontend (HTML, CSS, JavaScript, React JS)

Database Management System

- ❖ Mongo DB

Purpose:

Stores and manages users, properties, bookings, and payment records.

Database Infrastructure

Database triggers or application logic automatically update property status:

Available → when listed

Booked → when a booking is approved

Unavailable → when removed or expired

- ✓ This prevents double-booking and ensures real-time accuracy.

Data Protection During Downtime

Proposed System

The proposed Rental System is a web-based application that provides a centralized platform where landlords, tenants, and administrators can interact efficiently. All rental activities are handled digitally, reducing manual effort and errors.

The proposed system is an online, automated Rental System designed to overcome the limitations of the existing manual or semi-digital system (time-consuming work, errors, lack of security, no real-time updates, etc.).

Key Features of the Proposed System

1. Centralized Database

- ❖ All data (users, s, rentals, payments) stored in a single database
- ❖ Easy data retrieval and updates
- ❖ Reduces duplication and data loss

2. Online Listings

- ❖ Landlords can add, update, or remove details
- ❖ Real-time availability status
- ❖ Photos and detailed descriptions included

3. Easy Search

Tenants can search s by:

- Location
- Rent range
- type

Saves time compared to manual searching

User Authentication and Roles

- ❖ Secure login system
- ❖ Role-based access:
 - Admin
 - Landlord
 - Tenant

Rental Request Management

- ❖ Tenants send rental requests online
- ❖ Landlords approve or reject requests
- ❖ Automatic status updates

2.7 Online Payment System

- ❖ Digital rent payments
- ❖ Payment history stored
- ❖ Reduces cash handling risk

Table 4 How the Proposed System Solves Existing Problems

Existing problem	Proposed system solution
Time consuming process	Automated work flows
Error prone records	Digital database
No centralized data	Single integrated system
Difficult search	Advanced search filters
Limited security	Authentication & encryption
No real time updates	Authentication & encryption
High dependency on agents	Direct landlord–tenant interaction

4. System Architecture (Proposed)

- ❖ **Presentation Layer:** Web interface (browser-based)
- ❖ **Application Layer:** Business logic and processing
- ❖ **Database Layer:** Centralized non relational database

5. Advantages of the Proposed System

- ❖ Faster and more efficient rental process
- ❖ Improved accuracy and data consistency
- ❖ Reduced paperwork
- ❖ Enhanced security
- ❖ Better user experience
- ❖ Scalable for future enhancements

6. Future Enhancements (Optional)

- ❖ Mobile application
- ❖ Map-based search
- ❖ Automated rent reminders
- ❖ Rating and review system
- ❖ Chat system between landlord and tenant

- ✓ The proposed Rental System provides a modern, secure, and efficient solution to the problems of the existing rental infrastructure. By automating processes and centralizing data, it improves transparency, reduces dependency on agents, and simplifies renting for all users.

Chapter Three

Software Requirement Specification

3. Introduction

In today's fast-paced digital world, managing rentals using traditional manual methods has become inefficient and time-consuming. The increasing demand for rental properties requires a system that can handle property listings, tenant information, rental requests, and payments in an organized and secure manner. Manual processes often lead to data loss, errors, lack of transparency, and delayed communication between landlords and tenants.

The House Rental System is a web-based application designed to automate and simplify the rental process. It provides a centralized platform where landlords can list their properties, tenants can search and request s, and administrators can manage the system efficiently. The system ensures secure data handling, real-time availability updates, and improved communication among users.

This Software Requirement Specification (SRS) document describes the functional and non-functional requirements of the house Rental System. It serves as a reference for developers, testers, and stakeholders to understand the system's behavior, constraints, and performance expectations, ensuring the successful development and implementation of the application.

3.1 General Constraints

1. Internet Dependency

The system requires a stable internet connection for users to access listings, send rental requests, and make online payments.

2. Technology Constraints

the system must be developed using the selected technologies such as web browsers, server-side programming languages, and a relational database.

3. User Authentication Constraint

only registered users are allowed to access system features. Unauthorized users cannot perform rental or management activities.

4. Security Constraints

User data must be protected using authentication and encryption mechanisms. Access is restricted based on user roles (Admin, Landlord, and Tenant).

5. Payment Gateway Dependency

online payment functionality depends on the availability and reliability of third-party payment services.

6. Hardware Limitations

System performance depends on server capacity and user devices such as computers or mobile phones.

7. Legal and Policy Constraints

the system must comply with local rental laws and data protection regulations.

8. Scalability Constraints

System expansion depends on server resources and database capacity.

9. Browser Compatibility Constraint

the system is limited to supported web browsers and may not function properly on outdated versions.

3.3 Specific Requirements

3.2.1 External Interface Requirements

This section describes how the house Rental System interacts with users, hardware, software, and communication components.

System models

Table 5 User Management

Use Case Name	Use Case Identifier	Actor of the scenario
User registration	UC1	Guest(new user)
User log in	UC2	Registered tenant / land loard
Up date profile	UC3	Tenant/land loard

Table 6Property Management

Use Case Name	Use Case Identifier	Actor of the scenario
Add new house listing	UC4	Land loard
Edit/update house info	UC5	Land loard
Remove expired listing	UC6	Landloard/admin

Table 7 Search and Interaction

Use Case Name	Use Case Identifier	Actor of the scenario
Search and filter houses	UC7	Tenant
View house dataile	UC8	Tenant
Send rental request	UC9	Tenant

Table 8 Administration and Payments

Use Case Name	Use Case Identifier	Actor of the scenario
Verifiy new listing	UC10	Admin
Manage user accounts	UC11	Admin
Make online payment	UC12	Tenant

3.2.2 Use case Diagram

Use case diagrams are essential in developing this system for communication, requirements analysis, system design, testing, and project management. They visually

represent user interactions with a system, aiding in understanding system functionality and guiding the development process.

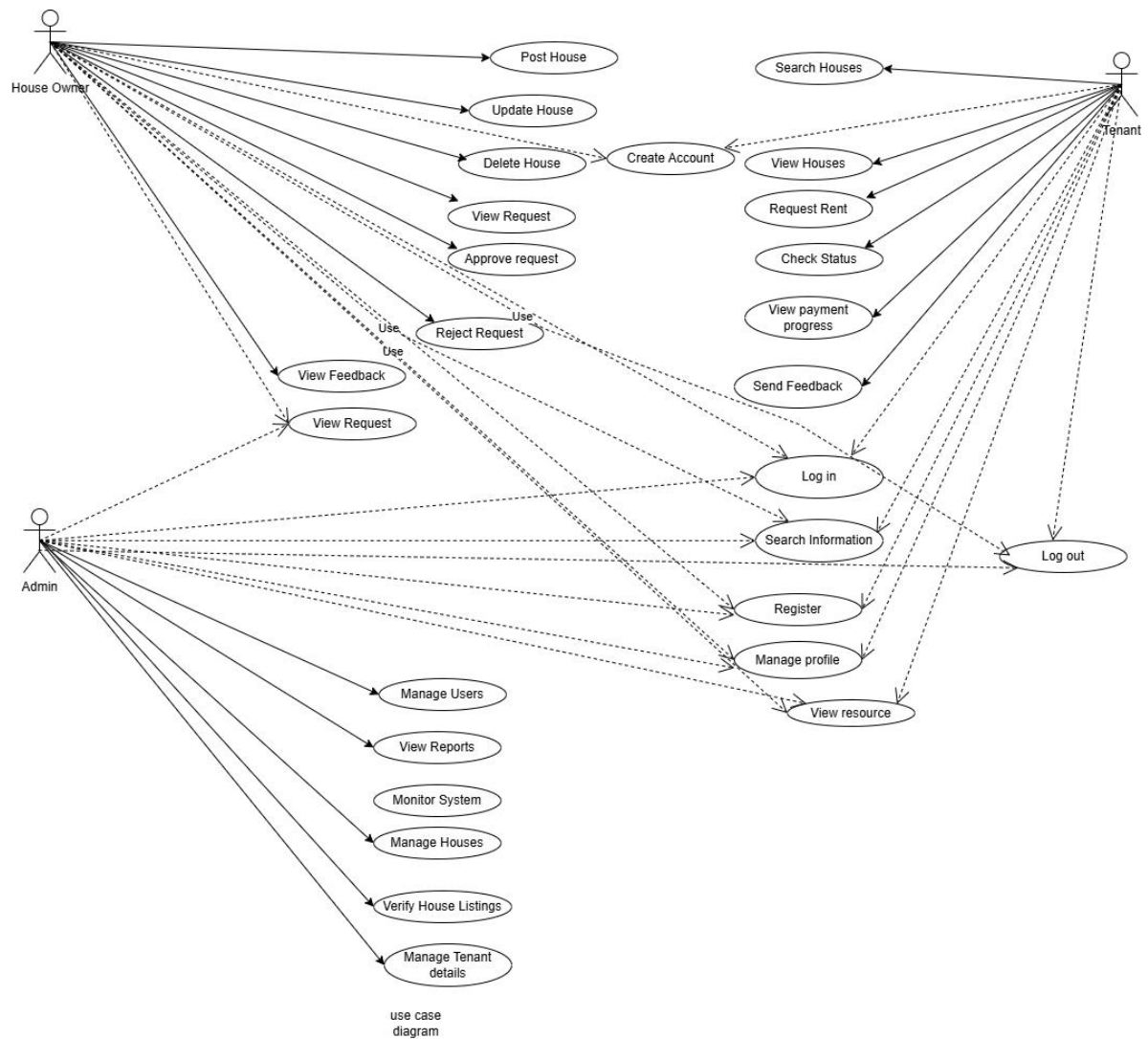


Figure 3:Use Case diagram

3.2.3 User Interfaces

- ❖ The system shall provide a **web-based graphical user interface**.
- ❖ Separate dashboards shall be provided for:
 - Admin
 - Landlord
 - Tenant

- ❖ The user interface shall include:
 - Login and registration pages
 - listing and search pages
 - Rental request management pages
 - Payment and history pages
- ❖ The interface shall be **user-friendly, consistent, and easy to navigate.**
- ❖ Error messages and confirmations shall be displayed clearly to users.

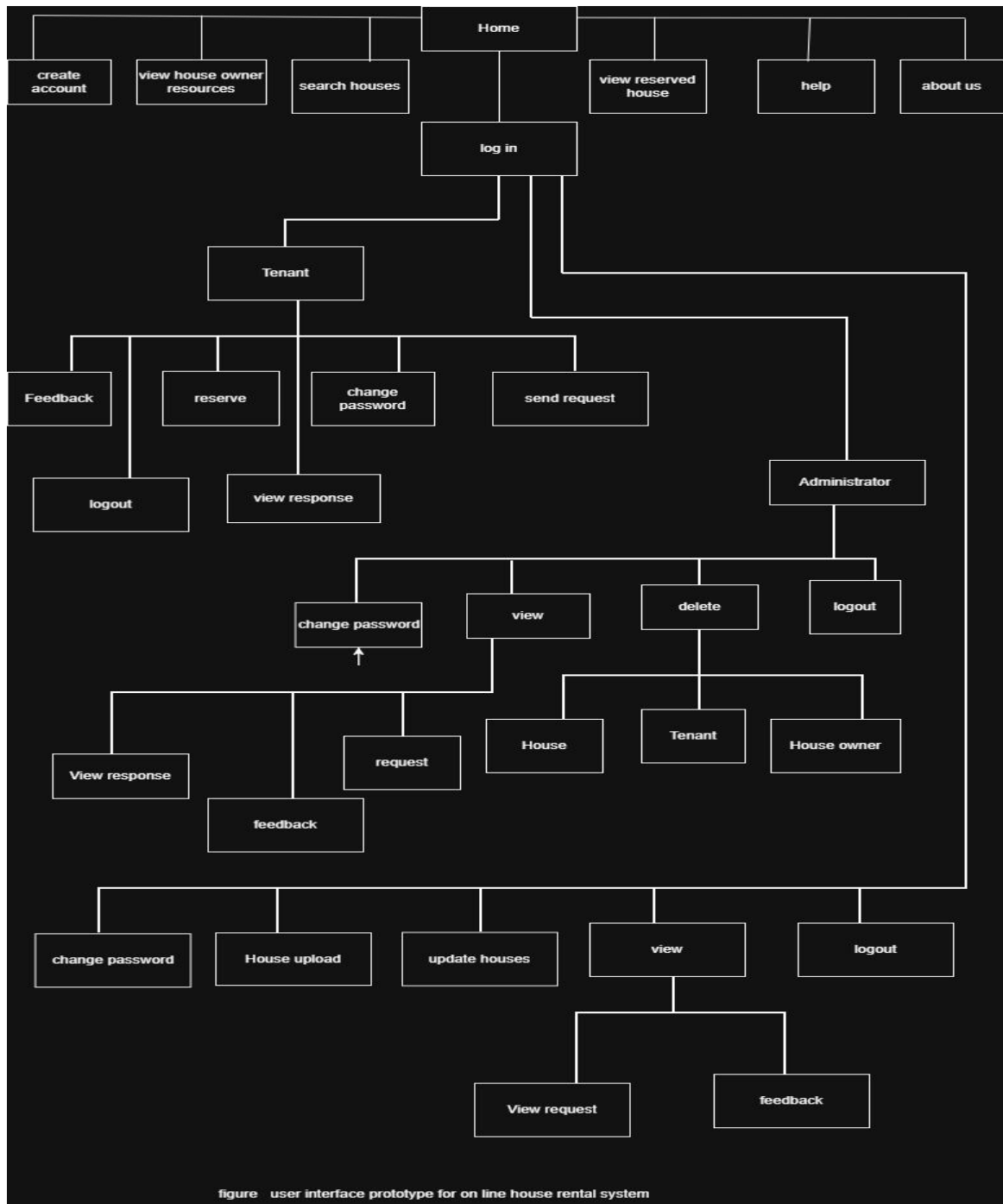


Figure 4:User Interface prototype diagram

3.2.4 Hardware Interfaces

- ❖ The system shall operate on standard **client devices** such as:
 - Desktop computers
 - Laptops
 - Smartphone's
- ❖ The system shall require a **server machine** to host the application and database.
- ❖ No special hardware devices are required to operate the system.
- ❖ The system shall interact with input devices such as keyboard, mouse, and touch screen.

3.2.4 Software Interfaces

- ❖ The system shall run on a **web server** such as Apache or Tomcat.
- ❖ The system shall use a **relational database** such as MySQL for data storage.
- ❖ The system shall be compatible with major web browsers:
 - Google Chrome
 - Mozilla Firefox
 - Microsoft Edge
- ❖ The system shall interface with a payment gateway for online transactions.
- ❖ The system shall support standard operating systems such as Windows and Linux.

3.2.5 Communications Interfaces

- ❖ The system shall use HTTP/HTTPS protocols for client–server communication.
- ❖ Secure communication shall be ensured using HTTPS.
- ❖ The system shall require an active internet connection.
- ❖ Notifications such as rental request status updates may be delivered through the web interface or email.
- ❖ Data transmission shall be reliable and secure.

3.3 Functional Requirements

3.3.1 User Registration and Authentication

- ❖ The system shall allow users to register as Admin, Landlord, or Tenant.
- ❖ The system shall validate user information during registration.
- ❖ The system shall authenticate users using a username and password.
- ❖ The system shall allow users to log out securely.
- ❖ The system shall restrict access based on user roles.

3.3.2 Listing and Management

- ❖ The system shall allow landlords to add new listings.
- ❖ The system shall allow landlords to update or delete details.
- ❖ The system shall display availability status.
- ❖ The system shall store details including location, rent, and description.

3.3.3 Search and Viewing

- ❖ The system shall allow tenants to search s by location, price, and type.
- ❖ The system shall display detailed information with images.
- ❖ The system shall show only available s to tenants.

3.3.4 Rental Request Management

- ❖ The system shall allow tenants to send rental requests.
- ❖ The system shall allow landlords to approve or reject rental requests.
- ❖ The system shall update rental status automatically.
- ❖ The system shall notify tenants about request status changes.

3.3.5 Payment Management

- ❖ The system shall allow tenants to make online payments.
- ❖ The system shall record payment details securely.
- ❖ The system shall generate payment receipts.

3.3.6 Admin Management and Reporting

- ❖ The system shall allow the admin to manage users and listings.
- ❖ The system shall allow the admin to view rental and payment reports.
- ❖ The system shall allow the admin to block or remove users if necessary.

3.4. Use Case Design

3.4.1. Actor Identification

Actors are users or external systems that interact with the House Rental System.

Table 10 Actor identification

Actor	Description
Tenant	A user who searches, views, and rents properties.
Property Owner	A user who lists s for rent and manages their properties.
Admin	System administrator who approves listings, monitors activity, and manages the system.
Payment Gateway	External system for processing online payments.

3.4.2. Use Case Identification

The main functionality of the system can be represented as the following use cases:

- Register Account
- Login
- List Property
- Search Property
- View Property Details
- Book Property
- Make Payment
- Manage Account

➤ Leave Review

3.4.3. Use Case #1: Register Account

Use Case Name: Register Account

Actors: Tenant, Property Owner

Description: Allows a new user to create an account to access the system.

Preconditions: User has internet access and is not already registered.

Post conditions: User account is successfully created.

Basic Flow:

- I. User opens the registration page.
- II. User enters personal details (name, email, password, contact info).
- III. User submits the registration form.
- IV. System validates data and creates an account.
- V. User receives confirmation of account creation.

Alternate Flow:

If email is already registered, system prompts an error message.

3.4.4. Use Case #2: List Property

Use Case Name: List Property

Actors: Property Owner

Description: Allows property owners to add their properties to the system for tenants to view and rent.

Preconditions: Owner is logged into the system.

Post conditions: Property is added to the system and pending admin approval.

Basic Flow:

- i. Owner logs into the system.
- ii. Owner navigates to “Add Property” page.
- iii. Owner enters property details (location, price, description, photos).
- iv. Owner submits the listing.

- v. System saves the property and sends it for admin approval.

3.4.5 Use Case Description

The use case descriptions provide detailed information about each functionality of the system:

Table 10 use case description

Use Case	Description
Register Account	Enables new users (tenant or owner) to create an account in the system.
Login	Authenticates users to access system features based on their role.
List Property	Allows owners to add new property listings for rent.
Search Property	Enables tenants to search available properties using filters like
View Property Details	Allows tenants to view detailed information of a selected property.
Book Property	Enables tenants to search available properties using filters like
Make Payment	Processes payments through the payment gateway to finalize bookings.
Approve Listing	Admin can approve or reject property listings submitted by owners.
Manage Account	Allows users to update personal information or account settings.
Leave Review	Tenants can submit reviews and ratings for rented properties.

3.5 Sequence Diagram

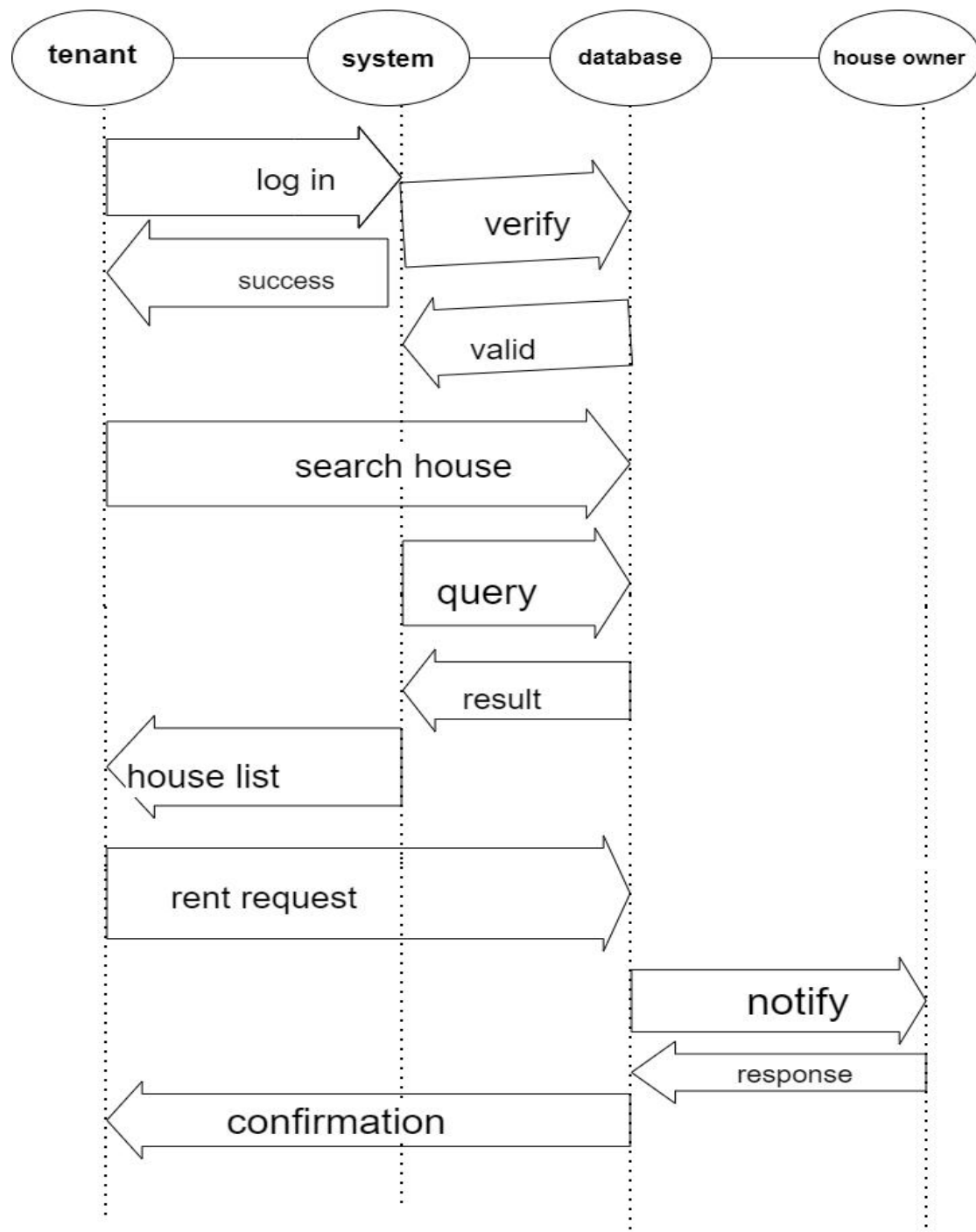


Figure 5:Sequence diagram

3.6 Activity diagram

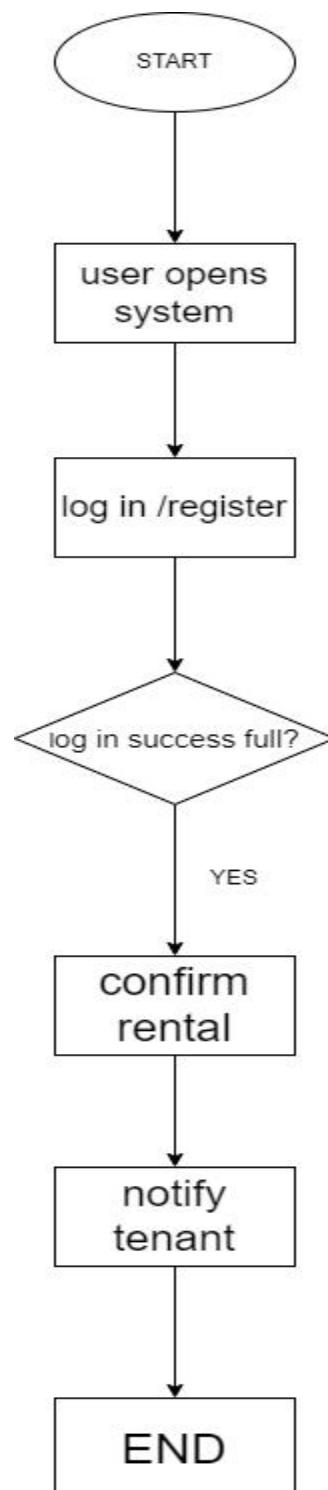


Figure 6:Activity diagram

3.8 Class diagram

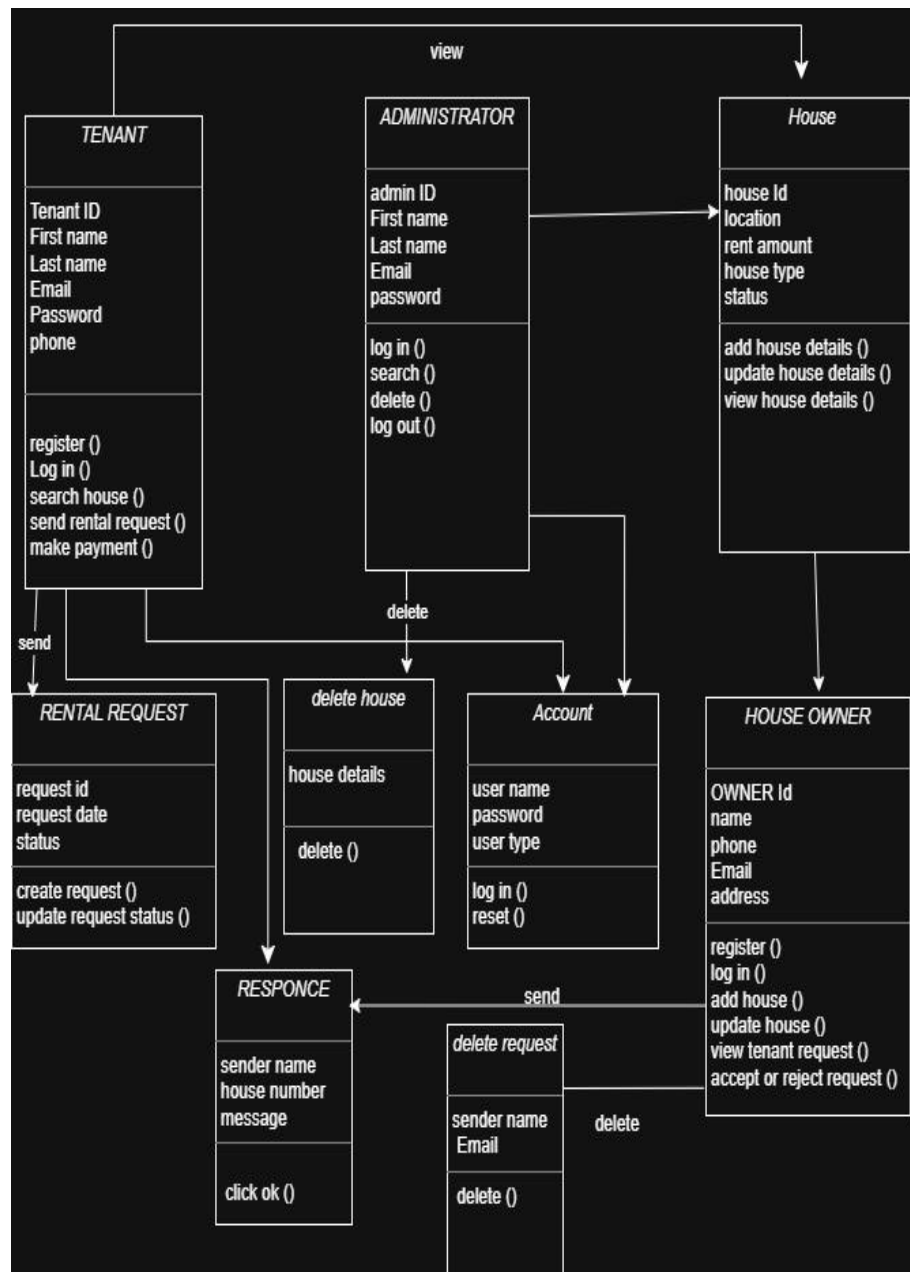


Figure 7:Class diagram

3.8.1 <class/object#1>

Class/Object #1: User

The **User** class represents all people who interact with the online house rental system.

This includes both tenants who search and rent s and owners who post their properties. The User class is important because almost every action in the system depends on user interaction.

Attributes

Use rid: used to uniquely identify each user in the system

Full Name: stores the real name of the user

Email: used for communication and login purposes

Phone Number: helps users contact each other when needed

Username: a unique name used during login

Password: stores the user's login password in encrypted form

Role: identifies whether the user is a tenant or a owner

Methods

Register (): allows a new user to create an account

Login (): enables the user to access the system

Update Profile(): allows the user to change personal information

Logout (): safely exits the system

This class ensures that only registered and authorized users can access the system features. It also helps in managing security and access control.

Class/Object #2:

The class represents rental properties listed in the system. Each belongs to a specific owner and contains all information needed by tenants to make rental decisions.

Attributes

ID: uniquely identifies each

Owner ID: links the to its owner

Title: short name or heading for the

Location: address or area where the is found

Type: type of (apartment, room, villa, etc.)

Number Of Rooms: indicates the size of the

price: rental cost

availability Status: shows whether the is available or rented

description: additional details about the

Methods

add(): allows owners to post new s

update(): enables editing of details

remove(): deletes a listing from the system

view(): displays details to users

The class helps manage rental listings and connects owners with potential tenants. It plays a key role in the searching and booking process.

Relationship between the Two Classes

One **User** (owner) can post multiple **s**, but each **s** belongs to only one owner. This shows a **one-to-many relationship** between the User and classes.

3.9 data structural model

The data structure model describes how data is organized, stored, and related within the system. It shows the main data objects used in the online house rental system and explains how these data items are connected. Proper data structuring is important to ensure efficient data storage, fast retrieval, and data consistency.

3.9.1 Entity -relation ship diagram

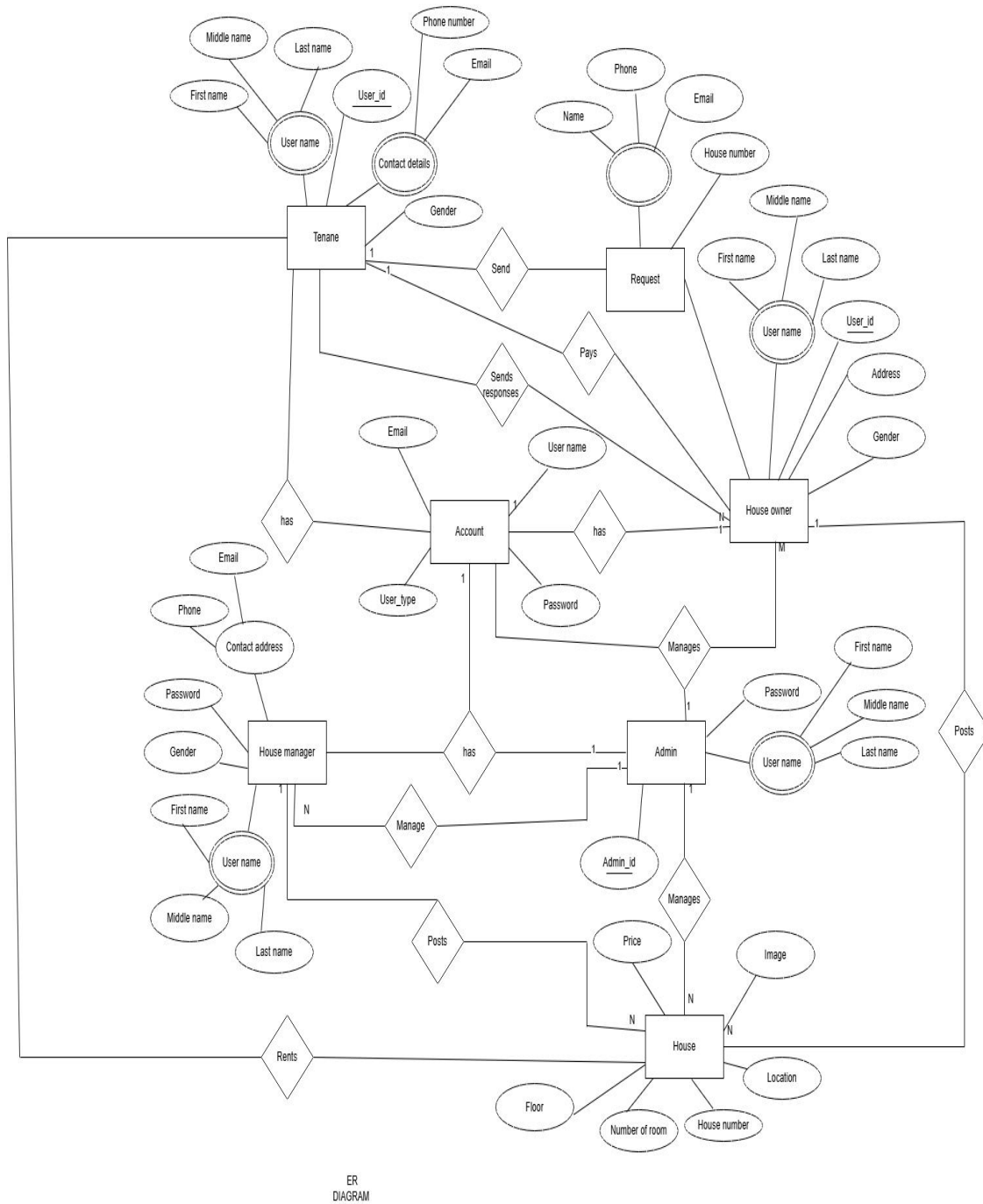


Figure 8: ER diagram

3.10.Non-Functional Requirements

The Rental System

Non-functional requirements describe how the system works rather than what the system does. These requirements focus on system quality, performance, and limitations. They are very important because they affect user satisfaction, system stability, and long-term usability.

3.10.1. Performance

The system should respond quickly when users search for s or open pages. Normal actions like login, search, and viewing details should not take much time. Even when many users are using the system at the same time, it should continue to work smoothly without hanging or crashing.

Performance refers to how fast and efficiently the system operates while users interact with it. The online rental system should provide quick responses to user actions such as login, searching for s, viewing details, and submitting rental requests. Pages should load within a reasonable time so that users do not feel frustrated or bored.

The system should be able to handle many users at the same time, especially during peak hours when many tenants are searching for s. Even under heavy usage, the system should continue working smoothly without slowing down. The database should process search queries and retrieve listings efficiently to avoid delays. Good performance helps increase user trust and encourages repeated use of the system.

3.10.2 Reliability

The system should work correctly and give accurate results at all times. User data such as information and bookings should be stored safely and should not be lost if an error occurs. If a problem happens, the system should recover easily without affecting users.

Reliability refers to the ability of the system to work correctly and consistently over time. The online house rental system should perform its functions accurately without unexpected failures. All data entered by users, such as details, rental requests, and user profiles, should be stored correctly and retrieved without errors.

The system should minimize crashes and system failures. If an error occurs due to network problems or server issues, the system should handle it properly and recover without losing important data. Reliable systems ensure that users can depend on the platform to manage rental information safely and correctly at all times.

3.10.3 Availability

The system should be available for use at any time of the day. Users should be able to access the system whenever they want, except during planned maintenance. System downtime should be very rare and kept as short as possible. Availability means that the system should be accessible whenever users need it. The online house rental system should be available 24 hours a day and 7 days a week so that users can search for s or manage their listings at any time.

System downtime should be very minimal and only occur during scheduled maintenance. When maintenance is required, it should be done at times when user activity is low. High availability is important because users may need to access the system urgently to find housing or respond to rental requests.

3.10.4 Security

The system should protect user information such as usernames, passwords, and contact details. Only authorized users should be allowed to access their accounts. The system should prevent unauthorized access, data misuse, and illegal activities.

Security is one of the most important non-functional requirements of the system. The online house rental system should protect user information such as usernames, passwords, phone numbers, and email addresses. Passwords should be stored securely so that they cannot be easily accessed or misused.

Only authorized users should be allowed to access their accounts and perform actions such as posting s or booking rentals. The system should prevent unauthorized access, hacking, and data theft. Proper authentication and access control mechanisms should be implemented to ensure that each user can only access their own information. A secure system builds user confidence and protects sensitive data.

3.10.5 Maintainability

The system should be easy to update and fix when problems occur. New features can be added in the future without affecting the existing system. The code should be clear and organized so that developers can understand and modify it easily. Maintainability refers to how easily the system can be updated, fixed, and improved. The online rental system should be designed in a way that allows developers to make changes without causing problems to the existing system. Bugs should be easy to identify and correct.

The system should support future enhancements such as adding new features, improving performance, or upgrading security. Clear system design and proper documentation help developers understand the system structure, making maintenance easier and more efficient. Good maintainability reduces long-term costs and ensures the system remains useful for many years.

3.10.6 Portability

The system should run properly on different devices and platforms. Users should be able to access the system using mobile phones, tablets, and computers. It should also work on different web browsers without any major issues. Portability refers to the ability of the system to operate on different platforms and Devices. The online rental system should be accessible from various devices such as desktop computers, laptops, tablets, and mobile phones.

The system should work properly on different web browsers without major compatibility issues. Users should not be restricted to a specific device or operating system. A portable system increases accessibility and allows more users to benefit from the platform regardless of their device type.

- ✓ Non-functional requirements play a vital role in the success of the online house rental system. Performance ensures fast responses, reliability guarantees correct operation, availability allows continuous access, security protects user data, maintainability supports future updates, and portability ensures system accessibility across different devices. Together, these requirements help create a stable, secure, and user-friendly system.

CHAPTER FOUR

SYSTEM DESIGN

4.1 Design Overview

System design is the process of transforming the requirements gathered during system analysis into a blueprint for system implementation. It defines the structure, components, interfaces, and data required to satisfy the specified requirements. The purpose of system design is to ensure that the proposed Online House Rental Management System is efficient, reliable, secure, maintainable, and user-friendly.

The Online House Rental Management System is designed as a web-based application that allows tenants, landlords, and administrators to interact through a centralized platform. The system automates the rental process by allowing landlords to post house listings, tenants to search and request rental properties, and administrators to supervise all activities performed within the system.

The design follows modern software engineering principles, including modularity, scalability, maintainability, and security. Each module of the system performs a specific task while communicating with other modules through well-defined interfaces. This approach makes the system easier to develop, test, maintain, and expand in the future.

The major design considerations include:

- Simplicity and ease of use
- Secure user authentication and authorization
- Efficient data management
- Fast retrieval of property information
- Reliable communication between users
- Responsive user interface
- Future scalability

The system is designed using a three-tier architecture consisting of the Presentation Layer, Application Layer, and Database Layer.

4.2 System Architectural Design

System architecture describes the overall structure of the software system and illustrates how different components interact with one another.

The architecture of the Online House Rental Management System is designed to ensure efficient processing of user requests while maintaining security and performance.

4.2.1 Chosen System Architecture

The system uses a Three-Tier Architecture.

1. Presentation Layer

The presentation layer is responsible for interacting with users. It provides graphical interfaces through which users perform various operations.

Functions include:

- User Registration
- User Login
- Property Search
- Property Listing
- Rental Requests
- Payment Interfaces
- Dashboard Access

Technologies Used:

- HTML
- CSS
- JavaScript
- React JS

Advantages:

- User-friendly interface
- Easy navigation
- Responsive design
- Platform independence

2. Application Layer

The application layer contains the business logic of the system. It processes user requests and communicates with the database layer.

Major modules include:

Authentication Module

Responsible for:

- User registration
- Login verification
- Password encryption
- Session management

Property Management Module

Responsible for:

- Adding properties
- Editing property information
- Deleting properties
- Viewing properties

Rental Request Module

Responsible for:

- Sending rental requests
- Accepting requests

- Rejecting requests
- Updating booking status

Payment Management Module

Responsible for:

- Processing online payments
- Recording payment information
- Generating receipts

Technologies Used:

- Node.js
- Express.js

3. Database Layer

The database layer stores and manages all system information.

The database stores:

- User information
- Property information
- Rental requests
- Payment records
- System logs

Technology Used:

- MongoDB

Benefits:

- Fast data retrieval
- Data consistency
- Scalability
- Security

Component Diagram Description

The major system components are:

User Interface Component

Provides interaction between users and the system.

Responsibilities:

- Accept user inputs
- Display results
- Display error messages

Authentication Component

Handles:

- Registration
- Login
- Password management

Property Management Component

Handles:

- Property creation
- Property updates
- Property deletion

Booking Component

Handles:

- Rental requests
- Booking approval
- Booking rejection

Payment Component

Handles:

- Online transactions
- Payment verification
- Receipt generation

Database Component

Stores all system information permanently.

Deployment Diagram Description

The deployment architecture consists of three major nodes:

Client Node

Contains:

- Web browser
- Mobile browser

Users access the system through these devices.

Web Server Node

Contains:

- Node.js server
- Application logic

Processes user requests.

Database Server Node

Contains:

- MongoDB database

Stores all data permanently.

Communication Flow:

Client Device → Web Server → Database Server

Responses follow the reverse direction.

4.2.2 System Interface Description

The Online House Rental Management System provides three major interfaces.

Tenant Interface

Allows tenants to:

- Register
- Login
- Search houses
- View house details
- Send rental requests
- Make payments

Landlord Interface

Allows landlords to:

- Register
- Login
- Add properties
- Update properties
- Remove properties
- Approve requests

Administrator Interface

Allows administrators to:

- Manage users
- Approve property listings

- Monitor activities
- Generate reports
- Remove invalid listings

4.3 Detailed Description of Components

4.3.1 Authentication Component

The Authentication Component is responsible for verifying user identity before granting access to the system.

Functions:

User Registration

The system collects:

- Full Name
- Email
- Phone Number
- Password
- User Role

The system validates all information before storing it.

User Login

The system verifies:

- Username
- Password

If valid, the user gains access to the system.

Password Security

Passwords are encrypted before being stored in the database.

Session Management

The system maintains active sessions and logs out inactive users automatically.

Benefits:

- Enhanced security
- User privacy
- Access control

4.3.2 Property Management Component

This component manages all house listings.

Functions:

Add Property

Landlords enter:

- House Title
- Location
- Rent Price
- Property Type
- Number of Rooms
- Description
- Images

Update Property

Landlords can modify existing listings.

Delete Property

Landlords may remove unavailable properties.

View Property

Users can browse all available properties.

Benefits:

- Centralized property management
- Accurate property information
- Easy updates

4.3.3 Rental Request Component

This component manages the rental process.

Functions:

Request Rental

Tenants submit rental requests.

Approve Request

Landlords review and approve requests.

Reject Request

Landlords reject unsuitable requests.

Status Update

The system automatically updates rental status.

Benefits:

- Organized rental process
- Real-time updates
- Reduced manual work

4.3.4 Payment Component

The payment component handles rent transactions.

Functions:

Make Payment

Tenants submit payment information.

Payment Verification

The system validates transactions.

Receipt Generation

Electronic receipts are generated.

Payment History

Users can view previous transactions.

Benefits:

- Secure transactions
- Reduced cash handling
- Better record management

4.4 User Interface Design

4.4.1 Description of User Interface

The user interface is designed to provide a simple and effective interaction environment.

Design principles used include:

1. Simplicity

Users can perform tasks with minimal effort.

2. Consistency

Similar screens have similar layouts.

3. Visibility

Important functions are clearly visible.

4. Feedback

The system provides immediate feedback for every action.

5. Error Prevention

Validation reduces incorrect data entry.

Home Page

Displays:

- Welcome message
- Featured properties
- Login button
- Registration button

Registration Page

Allows users to create accounts.

Login Page

Allows users to access the system.

Search Page

Provides search filters including:

- Location
- Rent Range
- Property Type

Property Details Page

Displays:

- Images
- Description
- Rent Amount
- Contact Information

Dashboard Pages

Separate dashboards exist for:

- Tenant
- Landlord
- Administrator

4.4.2 Screenshot Images of the system

1. Landing Page

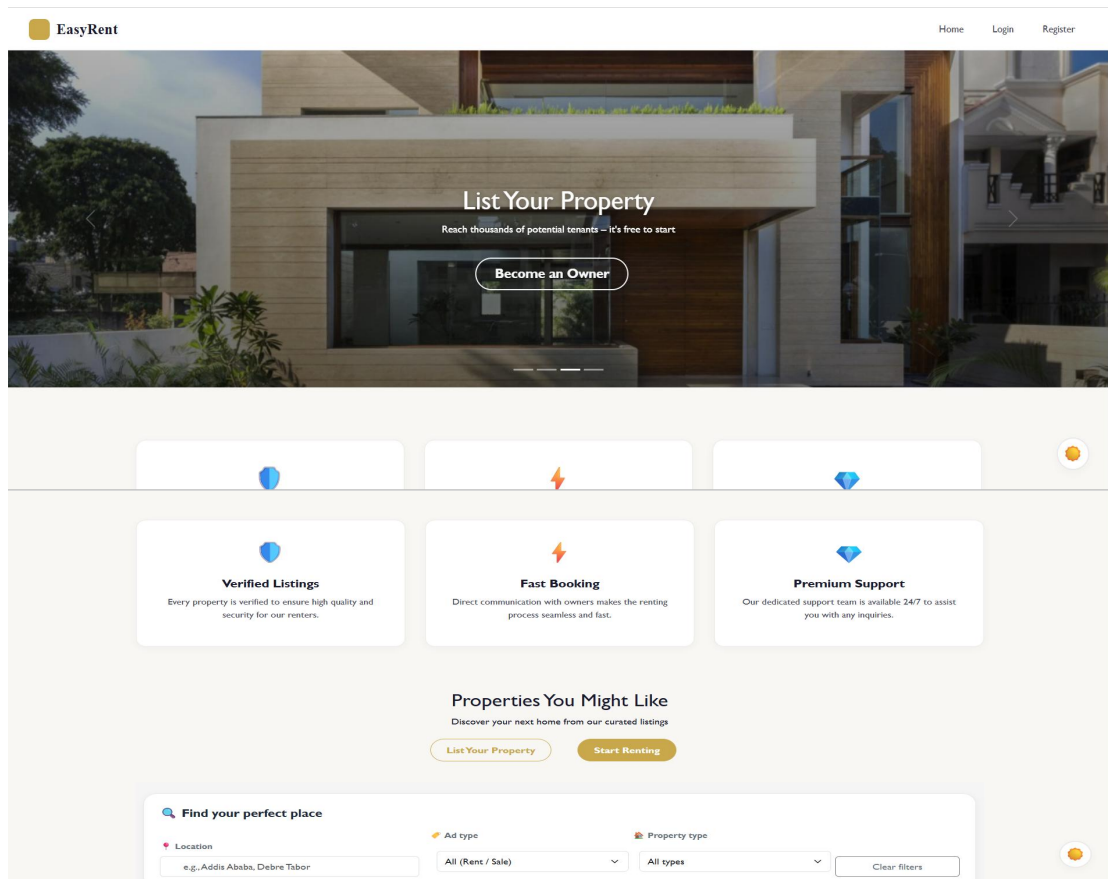
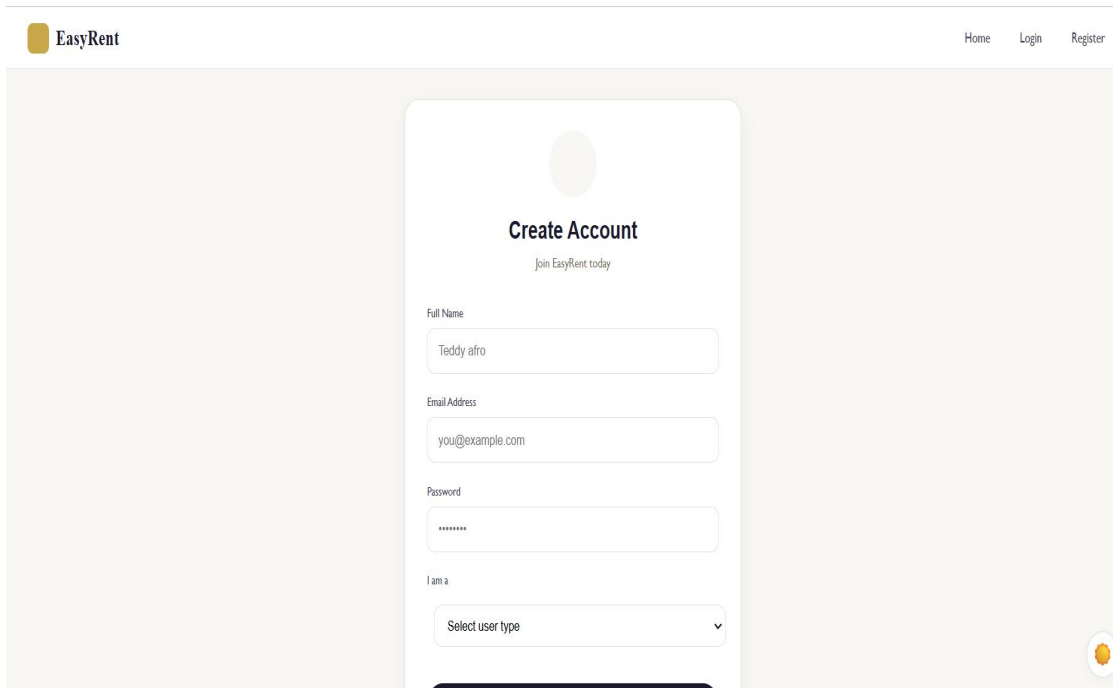


Figure 9:Landing page

2. Registration Page



The registration page features a central white card on a light beige background. At the top left of the card is the EasyRent logo, and at the top right are links for Home, Login, and Register. The card has a circular profile picture placeholder at the top. Below it, the heading "Create Account" is followed by the subtext "Join EasyRent today". The form includes fields for "Full Name" (containing "Teddy afro"), "Email Address" (containing "you@example.com"), and "Password" (masked with dots). Below the password field is a dropdown menu labeled "I am a" with the option "Select user type". A dark blue "Sign Up" button is at the bottom of the card. A small circular logo is in the bottom right corner of the page.

EasyRent

Home Login Register

Create Account

Join EasyRent today

Full Name

Teddy afro

Email Address

you@example.com

Password

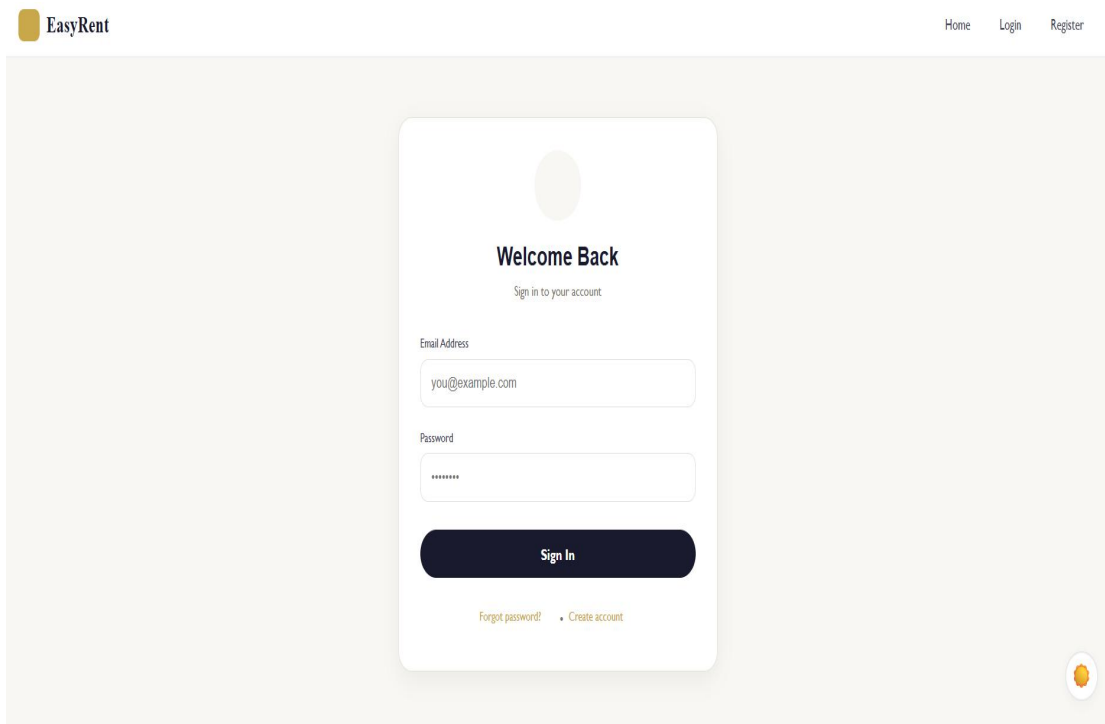
I am a

Select user type

Sign Up

Figure 10:Registration Page

3. Login Page



The login page features a central white card on a light beige background. At the top left of the card is the EasyRent logo, and at the top right are links for Home, Login, and Register. The card has a circular profile picture placeholder at the top. Below it, the heading "Welcome Back" is followed by the subtext "Sign in to your account". The form includes fields for "Email Address" (containing "you@example.com") and "Password" (masked with dots). Below the password field is a dark blue "Sign In" button. At the bottom of the card are links for "Forgot password?" and "Create account". A small circular logo is in the bottom right corner of the page.

EasyRent

Home Login Register

Welcome Back

Sign in to your account

Email Address

you@example.com

Password

Sign In

Forgot password? Create account

Figure 11:Login page

4. Tenant Dashboard

[illegible]

Figure 12: Tenant propert listing and request booking page

5. Landlord Dashboard

The screenshot shows the 'EasyRent' Landlord Dashboard. At the top, there's a navigation bar with 'Add Property', 'All Properties', and 'All Bookings'. The main section is titled 'PROPERTY MANAGEMENT' and 'List your property with ease'. Below this, there's a form to 'Fill in the details below to post your property for rent or sale. Reach thousands of potential tenants and buyers.' The form has several fields: 'PROPERTY TYPE' (Residential), 'LISTING TYPE' (For Rent), 'FULL ADDRESS' (e.g. 12 Maple Street, Lagos), 'PROPERTY IMAGES' (Click to upload images), 'PROPERTY DOCUMENTS' (Upload legal documents), 'CONTACT NUMBER' (0957874554), and 'AMOUNT (BIRR)' (0.00). There's a 'Log Out' button in the top right corner.

Figure 13:Propert owner page

6. Property Listing Page

The screenshot shows the 'Property Listing Page' on the 'EasyRent' website. At the top, there's a search bar with the text 'Find your perfect place'. Below the search bar, there are filters for 'Location' (e.g., Addis Ababa, Debre Tabor), 'Ad type' (All (Rent / Sale)), and 'Property type' (All types). There's a 'Clear filters' button. Below the filters, there's a section titled 'Available properties' with a '1 listing found' indicator. The listing is for 'Addis ababa' and is marked as 'Available'. The listing details include 'residential For rent', 'Addis ababa', 'Price: Br 12,000', and 'Contact: 988776655'. There's a 'Request booking' button at the bottom of the listing card.

Figure 14: Property Listing Page

7. Rental Request Page

The screenshot shows a modal window titled "Property details & booking request" with a close button (X) in the top right corner. The modal is divided into two main sections. The first section, "Property view 1", contains the following details: "For rent – residential", "Location: Addis ababa", "Price: Br 12,000", "Owner contact: 988776655", "Availability: Available" (with a green checkmark icon), and "Additional info: this is the detail of the building." The second section, "Booking request form", contains input fields for "Full name *" and "Phone number *", a "Government ID (Kebele, Passport, etc.) *" field with a "Choose File" button and "No file chosen" text, and a "Submit booking request" button. Below the ID field, there is a note: "Upload a clear photo or scan of your ID. Max 5MB (JPG, PNG, PDF)."

Property details & booking request

Property view 1

For rent – residential

Location: Addis ababa

Price: Br 12,000

Owner contact: 988776655

Availability: Available

Additional info: this is the detail of the building.

Booking request form

Full name *

Phone number *

Government ID (Kebele, Passport, etc.) *

Choose File No file chosen

Upload a clear photo or scan of your ID. Max 5MB (JPG, PNG, PDF).

Submit booking request

Figure 15:Rental requesting page

8. Payment Page

The screenshot shows a modal window titled "Booking Information" with a close button (X) in the top right corner. The modal displays booking details in a two-column layout. The left column contains: "BOOKING ID" (6a21d3bb307c2344a196102e), "CONTACT PHONE" (987755878), "PAYMENT" (pending), and "RENT AMOUNT" (Br 3,333). The right column contains: "APPLICANT NAME" (amha befirdu), "STATUS" (accepted), "PROPERTY ID" (6a21d35e307c2344a196101e), and "TRANSACTION ID" (TX-1780601861461-96102e). At the bottom right, there is a "Pay Now" button and a partially visible "Close" button.

Booking Information

BOOKING ID

6a21d3bb307c2344a196102e

CONTACT PHONE

987755878

PAYMENT

pending

RENT AMOUNT

Br 3,333

APPLICANT NAME

amha befirdu

STATUS

accepted

PROPERTY ID

6a21d35e307c2344a196101e

TRANSACTION ID

TX-1780601861461-96102e

Pay Now

Close

Figure 16:Payment Detail Page

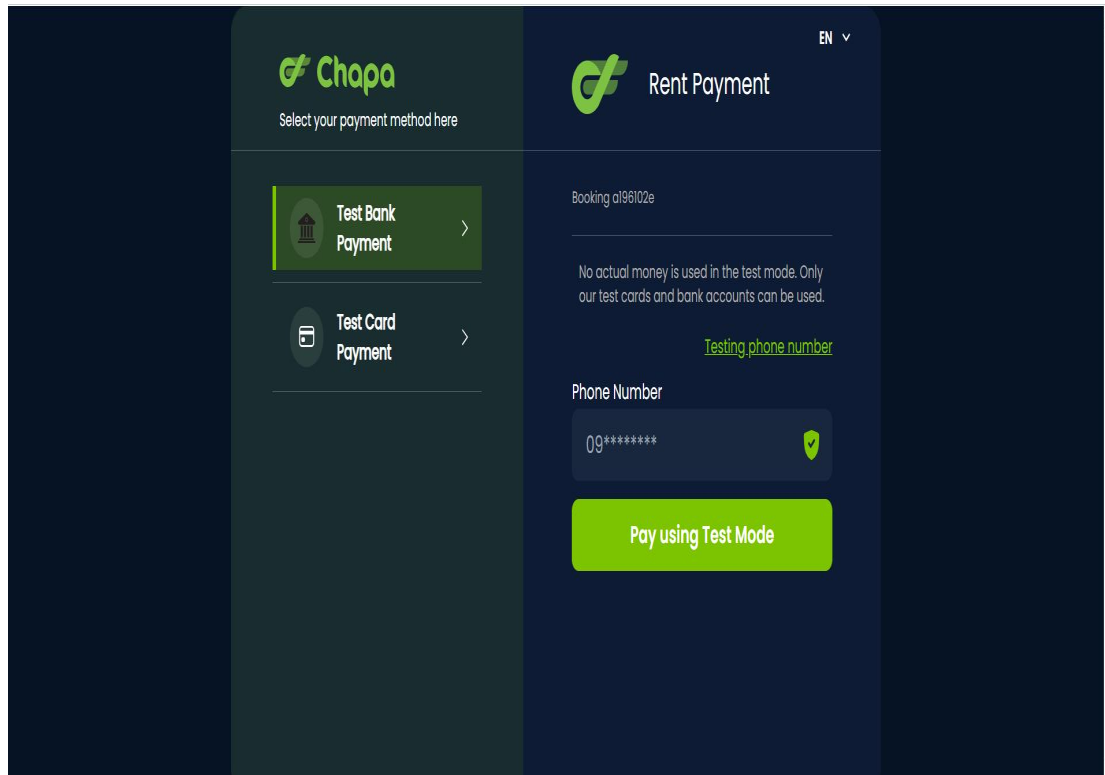


Figure 17: Chapa Payment page

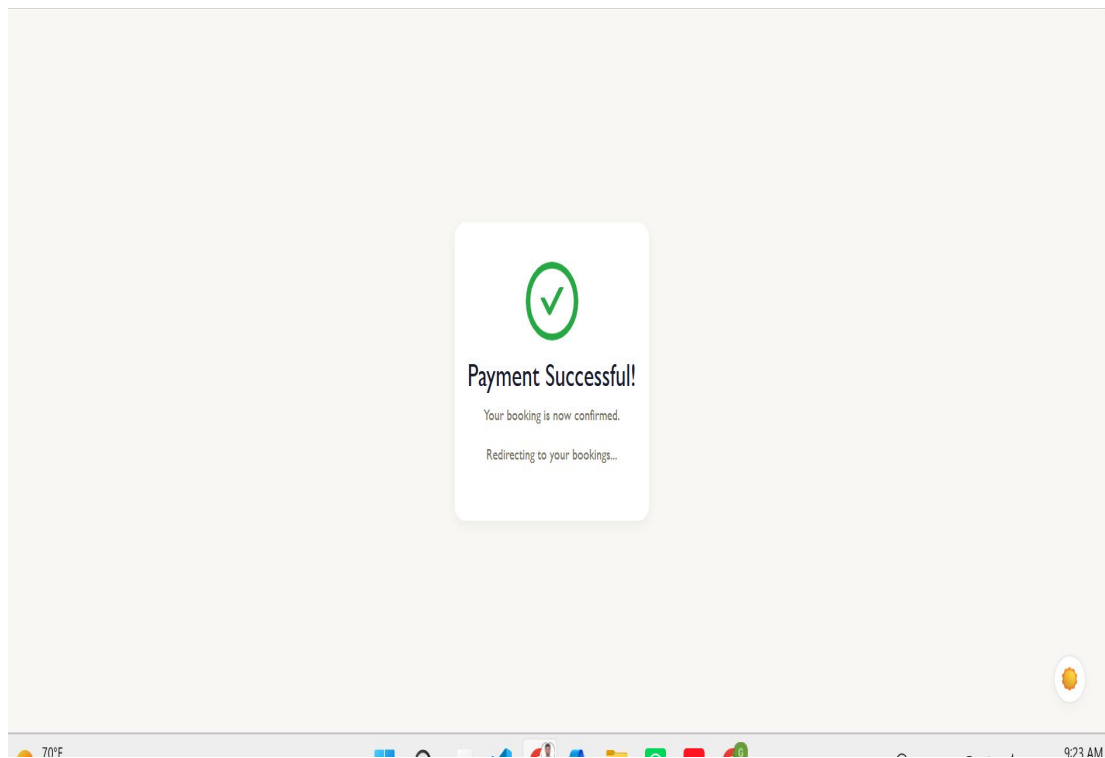


Figure 18: Payment status Page

9. Admin Dashboard

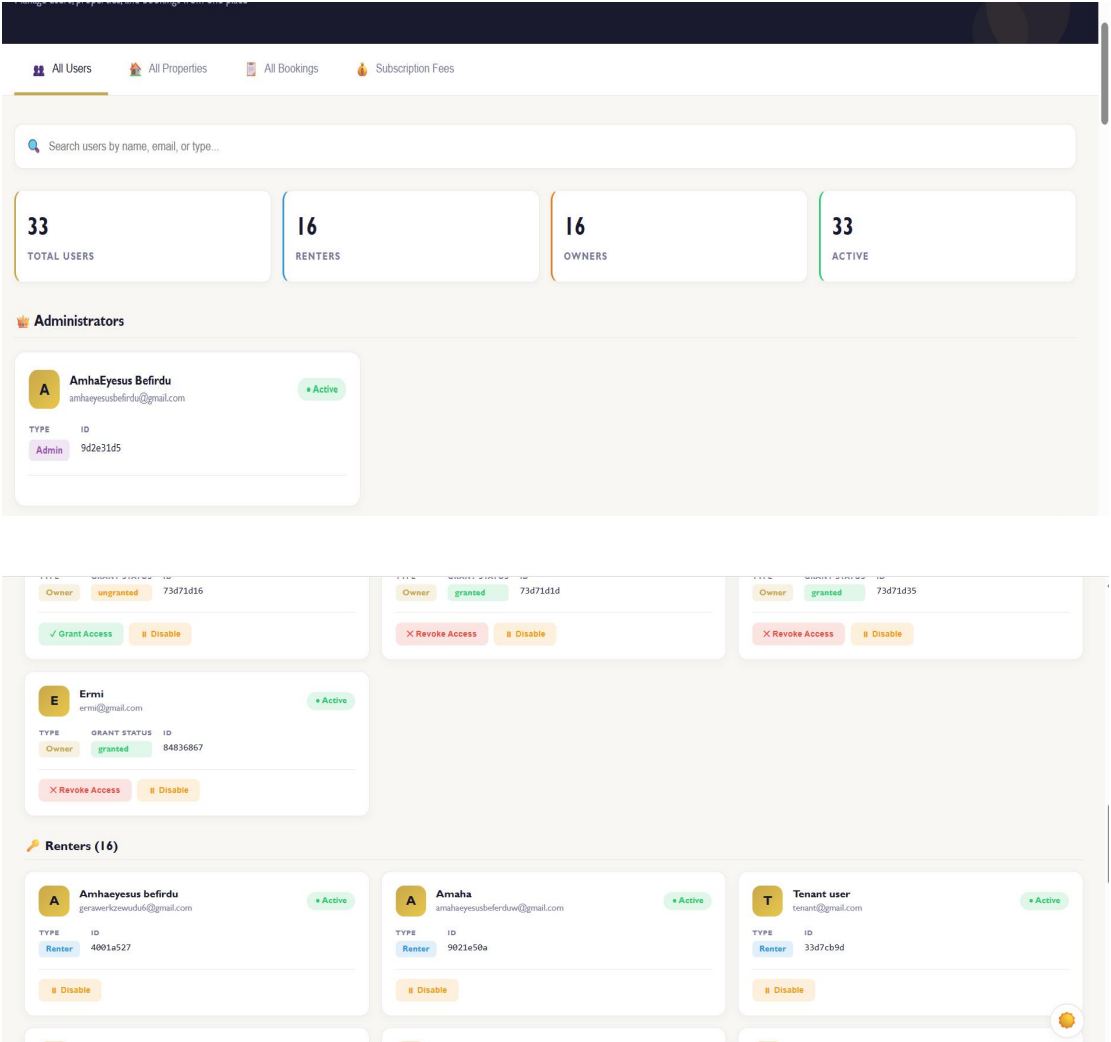


Figure 19:admin all User management page

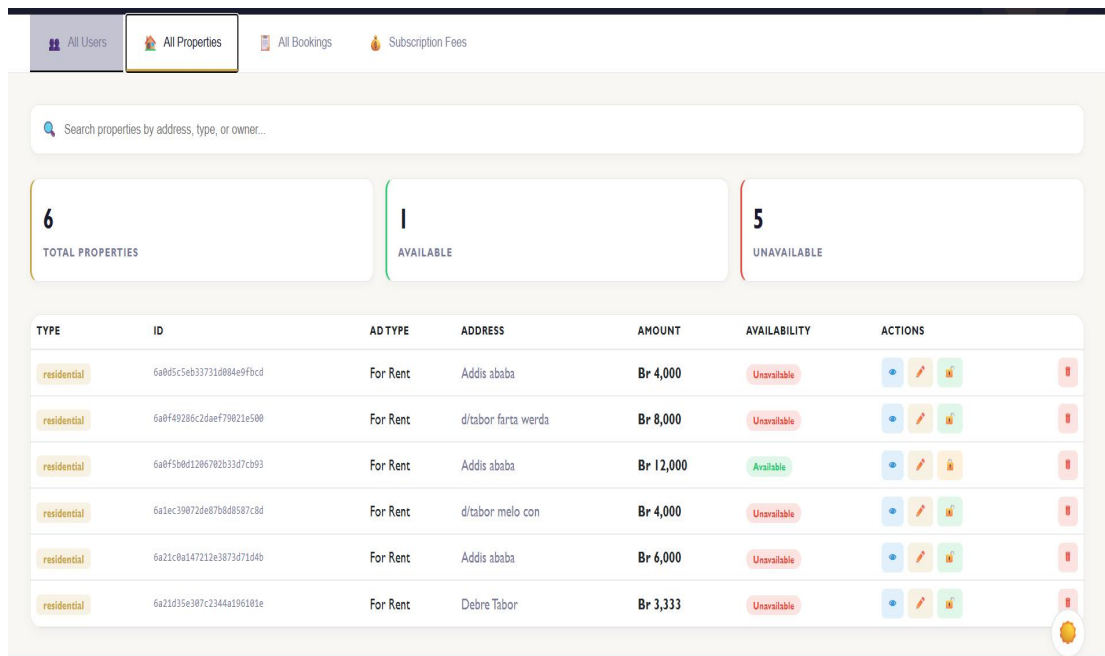


Figure 20:admin property approval page

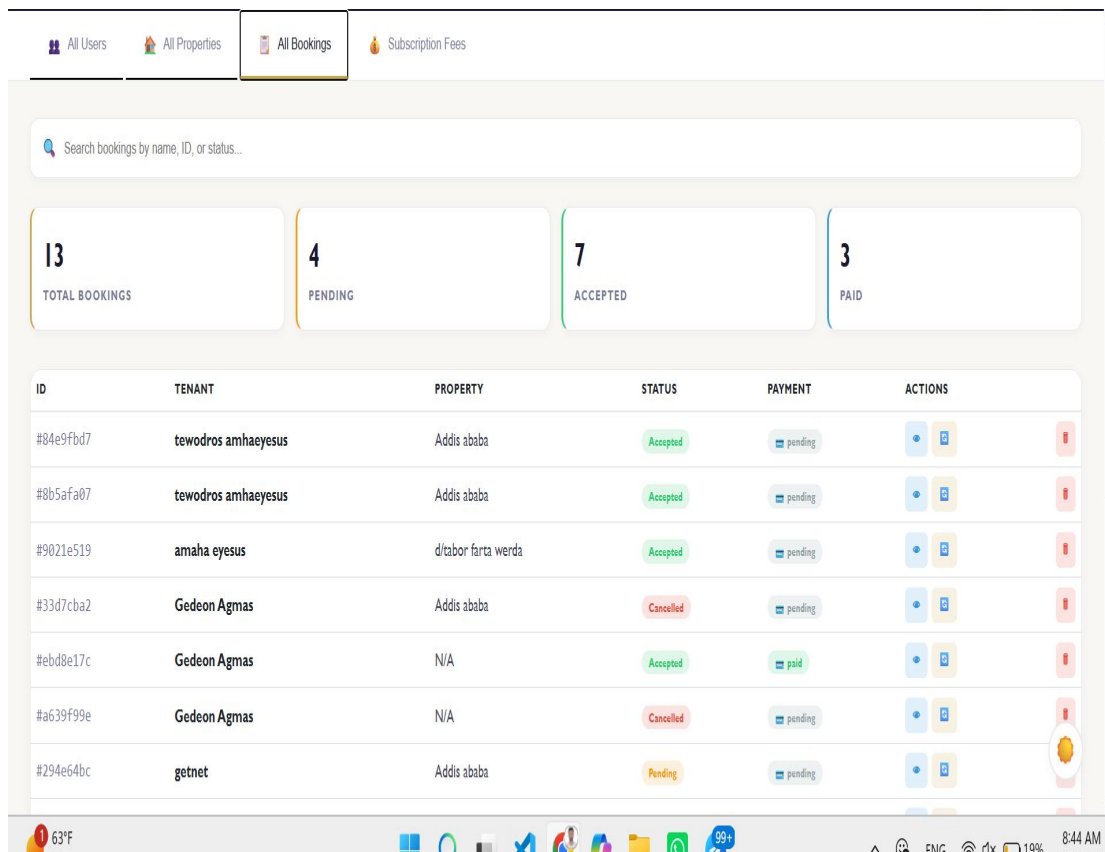


Figure 21:admin dashbourd for all booking summary

CHAPTER FIVE

SYSTEM TESTING

5.1 System Overview

System testing is one of the most important phases in software development. It is the process of evaluating and verifying that a software application performs according to its specified requirements. The main objective of testing is to identify defects, errors, and inconsistencies before the system is deployed to users.

For the Online House Rental Management System, testing was conducted to ensure that all modules operate correctly and that users can perform their intended tasks without difficulty. The testing process covered functional requirements, non-functional requirements, user interface behavior, security mechanisms, database operations, and system performance.

The major modules tested include:

- User Registration Module
- User Authentication Module
- Property Management Module
- Property Search Module
- Rental Request Module
- Payment Module
- Administrative Module

Testing was performed throughout the development process to ensure system quality, reliability, and usability.

5.2 Test Approach

The testing approach used in this project combines several testing methods to verify the correctness of the system.

The testing process was conducted in the following stages:

Unit Testing

Each module was tested independently to ensure that it performs its intended function correctly.

Examples:

- Registration form validation
- Login authentication
- Property upload functionality

Integration Testing

After unit testing, modules were combined and tested together.

Examples:

- Registration module connected with database
- Property module connected with search functionality
- Payment module connected with booking system

System Testing

The complete system was tested as a whole to verify that all components work together correctly.

User Acceptance Testing (UAT)

Potential users including tenants and landlords were invited to test the system and provide feedback.

Security Testing

Testing was conducted to verify:

- Password protection
- User authentication
- Access control

- Unauthorized access prevention

Performance Testing

The system was evaluated under multiple user requests to ensure acceptable response times.

5.3 Test Plan

The test plan describes what will be tested, how it will be tested, and the expected outcomes.

5.3.1 Features to be Tested

The following features were tested:

User Management

- User Registration
- User Login
- Password Validation
- Profile Update
- Logout

Property Management

- Add Property
- Edit Property
- Delete Property
- View Property

Property Search

- Search by Location
- Search by Price
- Search by Property Type

Rental Management

- Send Rental Request
- Approve Request
- Reject Request

Payment Management

- Online Payment
- Payment Confirmation
- Receipt Generation

Administrative Functions

- User Management
- Property Approval
- Report Generation.

5.3.2 Features Not to be Tested

The following features were excluded from testing because they belong to external systems:

- Internet Service Provider Operations
- Browser Internal Operations
- Third-Party Payment Gateway Internal Logic
- Operating System Services
- Hardware Components

5.3.3 Testing Tools and Environment

The following tools and environments were used.

Tool	Purpose
Google Chrome	Frontend Testing
Mozilla Firefox	Browser Compatibility Testing
Node.js	Backend Testing
MongoDB Compass	Database Testing

Postman	API Testing
Visual Studio Code	Development and Debugging

Hardware Environment

- Intel Core i5 Processor
- 8 GB RAM
- 256 GB Storage

Software Environment

- Windows 10/11
- Node.js
- React JS
- MongoDB
- Google Chrome

5.4 Test Cases

Test Case

Case Name

User Registration

Purpose

To verify that a new user can successfully register.

Inputs

- Full Name
- Email
- Phone Number
- Password

Expected Output

User account is created successfully.

Pass/Fail Criteria

Pass if account information is saved in the database.

Test Procedure

1. Open Registration Page.
2. Enter valid information.
3. Click Register button.

Test Result

Pass

5.5 Summary of Testing Results

The testing process confirmed that all major functionalities of the Online House Rental Management System operate correctly. The system successfully handled user registration, login authentication, property management, property searching, rental requests, online payments, and administrative functions. No critical defects were discovered during final testing. Minor user interface issues identified during testing were corrected before deployment.

The overall test results indicate that the system satisfies its functional and non-functional requirements and is ready for deployment.

CHAPTER SIX

CONCLUSION AND RECOMMENDATION

6.1 Conclusion

The Online House Rental Management System was developed to solve the challenges associated with traditional house rental processes. The existing system relied heavily on manual operations, paper-based records, physical visits, and informal communication methods, making the rental process slow, inefficient, and difficult to manage.

The proposed system provides a modern web-based platform that connects landlords and tenants through a centralized database. The system allows landlords to post property listings, tenants to search and request rental properties, and administrators to monitor all system activities. Through automation, the system significantly reduces manual work while improving efficiency, accuracy, transparency, and accessibility.

The system successfully achieved the objectives established at the beginning of the project. It provides secure user authentication, efficient property management, advanced property searching, rental request management, payment processing, and administrative control.

The testing results demonstrated that the system performs effectively and satisfies user requirements. The implementation of the Online House Rental Management System contributes to improving property rental management by providing a reliable, secure, and user-friendly environment for all stakeholders.

Therefore, it can be concluded that the developed system successfully addresses the limitations of the existing rental process and provides a practical solution for modern house rental management.

6.2 Recommendations

Based on the findings of this project, the following recommendations are proposed:

1. Property owners should adopt the system to improve the visibility of rental properties.
2. Tenants should utilize the platform to search and compare properties efficiently.
3. System administrators should perform regular database backups to prevent data loss.
4. Security updates should be applied regularly to protect user information.
5. Training should be provided for new users to improve system utilization.
6. Organizations implementing the system should maintain stable internet connectivity for better performance.
7. Regular system maintenance should be conducted to ensure reliability and availability.
8. User feedback should be collected continuously to improve future versions of the system.

6.3 Future Work

Although the Online House Rental Management System meets its current objectives, several improvements can be implemented in future versions.

Future enhancements may include:

- Mobile Application Development

Develop Android and iOS applications for easier access.

- GPS and Map Integration

Integrate interactive maps to help tenants locate properties more easily.

- Online Chat System

Allow direct communication between landlords and tenants within the platform.

- Review and Rating System

Enable tenants to review properties and landlords after transactions.

- Artificial Intelligence Recommendations

Recommend properties based on user preferences and search history.

- SMS and Email Notifications

Provide automatic notifications for booking approvals, payments, and updates.

- Digital Rental Agreements

Support electronic contracts and digital signatures.

- Advanced Analytics and Reporting

Provide detailed reports and dashboards for administrators and property owners.

- Multi-Language Support

Support multiple languages for broader accessibility.

➤ Cloud Deployment

Deploy the system on cloud infrastructure to improve scalability and availability. These future enhancements will further improve usability, efficiency, and overall user experience while making the system more competitive and technologically advanced.

References

- Agarwal, B., & Tayal, S. P. (2020). *Software engineering and testing*. Firewall Media.
- Bass, L., Clements, P., & Kazman, R. (2022). *Software architecture in practice* (4th ed.). Addison-Wesley.
- Dennis, A., Wixom, B. H., & Tegarden, D. (2020). *Systems analysis and design: An object-oriented approach with UML* (6th ed.). Wiley.
- Elmasri, R., & Navathe, S. B. (2017). *Fundamentals of database systems* (7th ed.). Pearson.
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine).
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Kendall, K. E., & Kendall, J. E. (2019). *Systems analysis and design* (10th ed.). Pearson.
- Larman, C. (2004). *Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development* (3rd ed.). Prentice Hall.
- MongoDB Inc. (2025). *MongoDB documentation*. <https://www.mongodb.com/docs>
- Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.
- React Team. (2025). *React documentation*. <https://react.dev>
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.
- Stallings, W. (2018). *Cryptography and network security: Principles and practice* (7th ed.). Pearson.

Welling, L., & Thomson, L. (2017). *PHP and MySQL web development* (5th ed.). Addison-Wesley.

World Wide Web Consortium (W3C). (2024). *HTML and CSS standards*.
<https://www.w3.org>

Node.js Foundation. (2025). *Node.js documentation*. <https://nodejs.org/en/docs>

Mozilla Developer Network. (2025). *Web development documentation*.
<https://developer.mozilla.org>

Oracle Corporation. (2024). *Database design concepts and administration*. Oracle Documentation.

Shelly, G. B., & Rosenblatt, H. J. (2012). *Systems analysis and design* (9th ed.). Cengage Learning.

Whitten, J. L., Bentley, L. D., & Dittman, K. C. (2004). *Systems analysis and design methods* (7th ed.). McGraw-Hill.